

author5hash=29db5b94c11f4e678669998ac7867043family=Alur, familyi=A., given=Rajeev, giveni=R.hash=bcf15a2a1b5e9adeabf29bf33f252acdfamily=familyi=D., given=Loris, giveni=L.hash=a72c89995414fe4ba7e530d7c0744e7afamily=Deshmukh, familyi=D., given=Jyotirmoy, giveni=J.hash=bbb784f797cfd0f00501e4654aa5cb465family=Raghothaman, familyi=R., given=Mukund, giveni=M.hash=e237862f206bd005ec2335744ce757familyi=Y., given=Yifei, giveni=Y. author2hash=22e757242574b20b1a043348c734d13fffamily=Balle, familyi=B., given=Borja, giveni=B.hash=8298c87c77cd46a5b2b7e799ba087945family=Mohri, familyi=M., given=Mehryar, giveni=M. author4hash=5e23c9abae9d9c24070dbb553b1b2210family=Basold, familyi=B., given=Henning, giveni=H.hash=a8088cc0a66020e35c2fd445e6222908family=Hansen, familyi=H., given=Helle Hvid, giveni=H. H.hash=a82e7dbce3953e8d8e09f1cdd82ebcfamilyi=P., given=Jean-Éric, giveni=J.-É.hash=371017393e6b1a7369b60fe9b3374cabfamily=Rutten, familyi=R., given=Jan, giveni=J. author2hash=bc42cf8bbdec81314e8afef2513f2ff5family=Bell, familyi=B., given=J. P., giveni=J. P.hash=d44bad97d1385c06efc0842e14b55187family=Smertnig, familyi=S., given=D., giveni=D. author2hash=a7c91ca1ae554a9377577c29716ae781family=Bell, familyi=B., given=Jason, giveni=J.hash=15ac0bfbaa4ef96bbe5aef453853b709family=Smertnig, familyi=S., given=Daniel, giveni=D. author4hash=8478322b6ec3952e5b20fb516dba2d0bfamily=Benedikt, familyi=B., given=M., giveni=M.hash=eb8362a209a8fd16f5a005d3e722d5abfamily=Duff, familyi=D., given=T., giveni=T.hash=4344a301605b6e287721c881b9c9familyi=S., given=A., giveni=A.hash=8902d81f585cae95e95497d1ca0e5549family=Worrell, familyi=W., given=J., giveni=J. author2hash=73e79a5714cbb6a302e5b703d7ca8a2cfamily=Bergeron, familyi=B., given=François, giveni=F.hash=8add4b0031d642844833104fe3d9a69afamilyi=R., given=Christophe, giveni=C. author2hash=73e79a5714cbb6a302e5b703d7ca8a2cfamily=Bergeron, familyi=B., given=François, giveni=F.hash=b713d6861c13da8e3060ee4f6078832efamily=Sattler, familyi=S., given=Ulrike, giveni=U. author2hash=a5a4d0ae306b7388a4705a59d0d34651family=Berstel, familyi=B., given=J., giveni=J.hash=c07bbe27f20214d7295da104c39aef1cfamily=Reutenauer, familyi=R., given=C., giveni=C. author4hash=7c4ebd50451e5b718eb3d5ad6cfbad71family=Blondel, familyi=B., given=Vincent D., giveni=V. D.hash=c2b5fc169871d0b556525fef63dfcaedfamilyi=J., given=Emmanuel, giveni=E.hash=279c0759f281679892ddfdcd1ded1dab7family=Koiran, familyi=K., given=Pascal, giveni=P.hash=26803570f296a87ed6859e67f24d75f5family=Portier, familyi=P., given=Natacha, giveni=N. author3hash=7fb6ba31261dbc6e32b2d250f75785f2family=Boreale, familyi=B., given=Michele, giveni=M.hash=8f78cf14908843578ef2a11bee312ba3family=familyi=C., given=Luisa, giveni=L.hash=cf849c81bee30a3677673e3c5c620224family=Gorla, familyi=G., given=Daniele, giveni=D. author2hash=7fb6ba31261dbc6e32b2d250f75785f2family=Boreale, familyi=B., given=Michele, giveni=M.hash=cf849c81bee30a3677673e3c5c620224family=Gorla, familyi=G., given=Daniele, giveni=D. author1hash=2192d83c2c342c54dd990b2acfb807dfamily=Bourbaki, familyi=B., given=Nicolas, giveni=N. author4hash=201447ab78fb900f4ab3d0965familyi=B.-M., given=Alex, giveni=A.hash=ee7a28458b5b02b2820720cea93b271bfamily=Cheval, familyi=C., given=Vincent, giveni=V.hash=a7c6d0cc8170736cde818de60584921cfamily=Shirmohammadi, familyi=S., given=Mahsa, giveni=M.hash=4225ab35a9e84541465345977a7f9c42family=Worrell, familyi=W., given=James, giveni=J. author5hash=475d74bf2c62b3199d0c6305877ed2f8family=Cadilhac, familyi=C., given=Michaël, giveni=M.hash=6bda235cded9dfd39a5d05438a685448familyi=M., given=Filip, giveni=F.hash=23d9ea1bf7134b69ce1a42bb4673e833family=Paperman, familyi=P., given=Charles, giveni=C.hash=1ab5cb4c1773d12f499567e578046996family=Pilipczuk, familyi=P., given=Michał, giveni=M.hash=e03b6ef520fd153bd8057523b58e36e0familyi=S., given=Géraud, giveni=G. author3hash=d1bdcc6d5faee9dc8204551fa4ee6f5afamily=Chen, familyi=C., given=Yu-Fang, giveni=Y.-F.hash=d369dc9fb63d777180c9182aac13f944family=Song, familyi=S., given=Lei, giveni=L.hash=4d88ff9753669ac5b145a6fdc4dce7e5family=Wu, familyi=W., given=Zhilin, giveni=Z. author3hash=369b82d5c9e19ede291c591e327a4cc3family=Chen, familyi=C., given=Kuo-Tsai, giveni=K.-T.hash=e993e6c4611a285eab95b66db13159e4familyi=F., given=Ralph H., giveni=R. H.hash=eb1ca9e41846af4bd0f3fdf6bad0bfe8family=Lyndon, familyi=L., given=Roger C., giveni=R. C. author1hash=937b34addddb902169a5176618a9269cfamily=Clemente, familyi=C., given=Lorenzo, giveni=L. author1hash=937b34addddb902169a5176618a9269cfamily=Clemente, familyi=C., given=Lorenzo, giveni=L. author1hash=937b34addddb902169a5176618a9269cfamily=Clemente, familyi=C.,

given=Lorenzo, giveni=L. author1hash=937b34addddb902169a5176618a9269cfamily=Clemente, familyi=C.,
 given=Lorenzo, giveni=L. author4hash=937b34addddb902169a5176618a9269cfamily=Clemente, familyi=C.,
 given=Lorenzo, giveni=L.hash=de18881ac270bfece1805788bfa15b58family=Donten-Bury, familyi=D.-B., given=Maria,
 giveni=M.hash=6bda235cded9dfd39a5d05438a685448family=Mazowiecki, familyi=M., given=Filip, giveni=F.hash=1ab5cb4c1773d12f499567e5
 familyi=P., given=Michał, giveni=M. author2hash=e8867c884d75d990c4e4407b71254420family=Coddington,
 familyi=C., given=Earl A., giveni=E. A.hash=383c8383f13be54ab69e28076ca08a38family=Levinson, familyi=L., given=Norman,
 giveni=N. author1hash=10a1ecca62e0d91a39d9cae9e315d1f4family=Cohn, familyi=C., given=Richard M.,
 giveni=R. M. author3hash=f491ad1431e56daa11842f8c7ad8aa73family=Cox, familyi=C., given=David A.,
 giveni=D. A.hash=8a72cf3903e931ba6c47c32f8d97825dfamily=Little, familyi=L., given=John, giveni=J.hash=7b115c829c11f9161b7d7d62ffa174
 familyi=O., given=Donal, giveni=D. author3hash=a55d494799e6922bea780f698aee311cfamily=D'Alessandro,
 familyi=D., given=Paolo, giveni=P.hash=768dee119bbfe62fb387e036f5541f12family=Isidori, familyi=L., given=Alberto,
 giveni=A.hash=395342fd685b9e33599faac8a4e2289afamily=Ruberti, familyi=R., given=Antonio, giveni=A. au-
 thor2hash=08db5faaf14779bd520b24f44574c976family=Denef, familyi=D., given=Jan, giveni=J.hash=0e4828039743e8df5d495fa062503f8bfamil
 familyi=L., given=Leonard, giveni=L. editor3hash=4c665fe871eb2a54cc4a531402fd0d13bfamily=Droste, fam-
 ilyi=D., given=Manfred, giveni=M.hash=a961d70f84130b5ac1e370e3d9d43692family=Kuich, familyi=K., given=Werner,
 giveni=W.hash=eaf7b92a6b7eba541f13ea4cd2d975b6family=Vogler, familyi=V., given=Heiko, giveni=H. au-
 thor2hash=3bd141eeb3527f562465698074a72954family=Eilenberg, familyi=E., given=Samuel, giveni=S.hash=ee7e692312c310fd33fbfdd5f0ce27
 familyi=L., given=Saunders Mac, giveni=S. M. author1hash=aae57fe3f3726fcbced8c287c38380ecfamily=Esparza,
 familyi=E., given=Javier, giveni=J. author1hash=7390174061fd92d49b138f3166d3d795family=Farzan,
 familyi=F., given=Azadeh, giveni=A. author1hash=e80e2252f0cbf7d74e5163507d0c8f31family=Fließ,
 familyi=F., given=Michel, giveni=M. author1hash=e80e2252f0cbf7d74e5163507d0c8f31family=Fließ,
 familyi=F., given=Michel, giveni=M. author1hash=e80e2252f0cbf7d74e5163507d0c8f31family=Fließ,
 familyi=F., given=Michel, giveni=M. author1hash=e80e2252f0cbf7d74e5163507d0c8f31family=Fließ,
 familyi=F., given=Michel, giveni=M. author1hash=e993e6c4611a285eab95b66db13159e4family=Fox,
 familyi=F., given=Ralph H., giveni=R. H. author1hash=4801fa2514dc070193cae9625806c5family=Grabolle,
 familyi=G., given=Gustav, giveni=G. author2hash=1fd337266f852975ab20346bb566ac42family=Huang,
 familyi=H., given=Feidan, giveni=F.hash=eb83518c8c7a0f403c681e530ffeee30family=Cao, familyi=C., given=Fasheng,
 giveni=F. author3hash=bd269b58701184b14b67cc8d3c428d58family=Jecker, familyi=J., given=Ismaël,
 giveni=I.hash=6bda235cded9dfd39a5d05438a685448family=Mazowiecki, familyi=M., given=Filip, giveni=F.hash=d3be929692e4670f05c987984
 familyi=P., given=David, giveni=D. author1hash=f21a937cde34af7b3489bde1d52b0b2ffamily=Kaplansky,
 familyi=K., given=Irving, giveni=I. author1hash=0c748d0ce016c59a4b93d61249b5355afamily=Karhumäki,
 familyi=K., given=Juhani, giveni=J. author1hash=0c748d0ce016c59a4b93d61249b5355afamily=Karhumäki,
 familyi=K., given=Juhani, giveni=J. author1hash=a8cf3ce0caae1e2e4fa47b13aee4b13cfamily=Kolchin,
 familyi=K., given=E. R., giveni=E. R. author1hash=6f419d439b61a2a6a0c0192b0e0db3fcfamily=Kostolányi,
 familyi=K., given=Peter, giveni=P. author2hash=6f419d439b61a2a6a0c0192b0e0db3fcfamily=Kostolányi,
 familyi=K., given=Peter, giveni=P.hash=8d0adfb8838ac873626fb57af79202c8family=Mišún, familyi=M., given=Filip,
 giveni=F. author2hash=a961d70f84130b5ac1e370e3d9d43692family=Kuich, familyi=K., given=Werner,

giveni=W.hash=27586e2da87e1abd2dedf337cf78bb12family=Salomaa, familyi=S., given=Arto, giveni=A. au-
 thor1hash=49d046cebe26f686f255b8a96cece7efamily=Levin, familyi=L., given=Alexander, giveni=A. au-
 thor1hash=0e4828039743e8df5d495fa062503f8bfamily=Lipshitz, familyi=L., given=Leonard, giveni=L. au-
 thor1hash=a7480b4216cb1b22565527af2d8642c5family=Litow, familyi=L., given=B., giveni=B. au-
 thor1hash=a82ae44fde79a45c4d7554a3fff33ee9family=Lopez, familyi=L., given=Aliaume, giveni=A. au-
 thor1hash=afa7a85697739666224f84996ac89dfffamily=Lothaire, familyi=L., given=M., giveni=M. au-
 thor1hash=43ae1bf2f98266ff2997572eead07176family=Matijasevič, familyi=M., given=Ju. V., giveni=J. V. au-
 thor1hash=cac50db2a50745fbfff26230282b557family=Mayr, familyi=M., given=Ernst, giveni=E. au-
 thor1hash=6191001c704a15f11454348698fc2663family=Mayr, familyi=M., given=Ernst W., giveni=E. W. au-
 thor3hash=8298c87c77cd46a5b2b7e799ba087945family=Mohri, familyi=M., given=Mehryar, giveni=M.hash=3ef139cfd819ad7d3af54f3d2e9507b9family= familyi=P., given=Fernando, giveni=F.hash=25433f7325fdf9811b6d237284481232family=Riley, familyi=R., given=Michael, giveni=M. author2hash=06976483cb5ccbd8d005ff266bda96bfffamily=Nasu, familyi=N., given=Masakazu, giveni=M.hash=379c2d2bfa5faf70282d2cd81661a2eafamily=Honda, familyi=H., given=Namio, giveni=N. au-
 thor2hash=c72ed22a294a9144f30309570101a352family=Novikov, familyi=N., given=Dmitri, giveni=D.hash=df1396d441584227959aabfb9fd52532family= familyi=Y., given=Sergei, giveni=S. author2hash=3d2d1d8687de3baa625d3d62a7e0e464family=Oppenheim, familyi=O., given=Alan V., giveni=A. V.hash=fd563e605c810572cece256b4271abd4family=Schafer, familyi=S., given=Ronald W., giveni=R. W. author1hash=69f39c41e80eec5978de9424afcff0b5family=Paz, familyi=P., given=Azaria, giveni=A. author3hash=d9d090f834c92fa1a75b5ce705bb2f61family=Pemantle, familyi=P., given=Robin, giveni=R.hash=73922b4b866b9840f6d45d95ddebfa8efamily=Wilson, familyi=W., given=Mark C., giveni=M. C.hash=37e17bdd0155a572f7934837592728c5 familyi=M., given=Stephen, giveni=S. author3hash=e7389c7d5224db72299a5d5e8d250f8dfamily=Pogudin, familyi=P., given=Gleb, giveni=G.hash=f20717fca657847f0bcbaf8093fe9d36family=Scanlon, familyi=S., given=Thomas, giveni=T.hash=82617a3c1734f7fc499d2631ae09c0b2family=Wibmer, familyi=W., given=Michael, giveni=M. au-
 thor1hash=8add4b0031d642844833104fe3d9a69afamily=Reutenauer, familyi=R., given=Christophe, giveni=C. au-
 thor1hash=8add4b0031d642844833104fe3d9a69afamily=Reutenauer, familyi=R., given=Christophe, giveni=C. au-
 thor1hash=ddf0111a239e4ea5a18096410aa2ca79family=Ritt, familyi=R., given=Joseph Fels, giveni=J. F. au-
 thor1hash=33be2fc2739d3493c6d6cc5d56a4c41efamily=Rudin, familyi=R., given=Walter, giveni=W. au-
 thor1hash=7abefe7a5693efc7fa98b424155f6c42family=Schützenberger, familyi=S., given=Marcel Paul, giveni=M. P. au-
 author1hash=e03b6ef520fd153bd8057523b58e36e0family=Sénizergues, familyi=S., given=Géraud, giveni=G. au-
 thor1hash=a61a7c4d8f4b467437038b8bc2aee208family=Stanley, familyi=S., given=Richard P., giveni=R. P. au-
 thor1hash=f216db3c75ba7e4acba4d76f9d21df75family>Wechler, familyi=W., given=Wolfgang, giveni=W. au-
 thor1hash=9cba74c03d2a5efde985e575dab99466family=Wilf, familyi=W., given=Herbert S., giveni=H. S. au-
 thor1hash=96d80a8e65b1c29cb1c6faac4c31d3befamily=Zeilberger, familyi=Z., given=Doron, giveni=D.

The commutativity problem for effective varieties of formal series, and applications

LORENZO CLEMENTE*, Dept. of Mathematics, Informatics and Mechanics, University of Warsaw, Poland

A *formal series* in noncommuting variables Σ over the rationals is a mapping $\Sigma^* \rightarrow \mathbb{Q}$ from the free monoid generated by Σ to the rationals. We say that a series is *commutative* if the value in the output does not depend on the order of the symbols in the input. The *commutativity problem* for a class of series takes as input a (finite presentation of) a series from the class and amounts to establishing whether it is commutative. This is a very natural, albeit nontrivial problem, which has not been considered before from an algorithmic perspective.

We show that commutativity is decidable for all classes of series that constitute a so-called *effective prevariety*, a notion generalising Reutenauer’s varieties of formal series. For example, the class of *rational series*, introduced by Schützenberger in the 1960’s, is well-known to be an effective (pre)variety, and thus commutativity is decidable for it.

In order to showcase the applicability of our result, we consider classes of formal series generalising the rational ones. We consider *polynomial automata*, *shuffle automata*, and *infiltration automata*, and we show that each of these models recognises an effective prevariety of formal series. Consequently, their commutativity problem is decidable, which is a novel result. We find it remarkable that commutativity can be decided in a uniform way for such disparate computation models.

Finally, we present applications of commutativity outside the theory of formal series. We show that we can decide *solvability* in sequences and in power series for restricted classes of algebraic difference and differential equations, for which such problems are undecidable in full generality. Thanks to this, we can prove that the syntaxes of multivariate *polynomial recursive sequences* and of *constructible differentially algebraic power series* are effective, which are new results which were left open in previous work.

CCS Concepts: • **Theory of computation** → **Formal languages and automata theory**; **Quantitative automata**.

Additional Key Words and Phrases: weighted automata, formal series, commutativity problem

ACM Reference Format:

Lorenzo Clemente. 2025. The commutativity problem for effective varieties of formal series, and applications. 1, 1 (October 2025), 41 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

A *univariate formal series* over the rationals is a function $\mathbb{N} \rightarrow \mathbb{Q}$ (sometimes written $\mathbb{Q}[[x]]$). They arise in a variety of disciplines, e.g., Taylor series in mathematical analysis [Rudin:Analysis:1976], generating functions in combinatorics [Stanley:1986, Wilf:generatingfunctionology:2005], and Z-transforms in engineering [OppenheimSchafer:DSP:1975]. The denomination “formal” highlights the algebraic aspect, rather than the analytic one; for brevity, we will refer to formal series simply as series.

*We acknowledge partial support of the NCN Sonata Bis grant 2024/54/E/ST6/00287 and of the ERC Starting grant INFSYS, agreement no. 950398.

Author’s Contact Information: Lorenzo Clemente, clementelorenzo@gmail.com, Dept. of Mathematics, Informatics and Mechanics, University of Warsaw, Warszawa, Poland.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

Manuscript submitted to ACM

Manuscript submitted to ACM

The theory of univariate series can be developed in several directions. One direction is more popular in mathematics, and considers *multivariate series in commuting indeterminates* $X = \{x_1, \dots, x_d\}$, which are functions of the form $\mathbb{N}^d \rightarrow \mathbb{Q}$ for some $d \in \mathbb{N}_{\geq 1}$ (sometimes written $\mathbb{Q}[[X]]$) [PemantleWilsonMelczer:CUP:2024]. Another direction is more popular in theoretical computer science, and considers *multivariate series in noncommuting indeterminates* $\Sigma = \{a_1, \dots, a_d\}$, which are functions of the form $\Sigma^* \rightarrow \mathbb{Q}$ (sometimes called *weighted languages* and written $\mathbb{Q}\langle\langle\Sigma\rangle\rangle$) [BerstelReutenauer:CUP:2010]. In fact, noncommuting indeterminates are more general, since we can recover commuting indeterminates as a special case: Namely, as those series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ which are *commutative*, i.e., $f(u) = f(v)$ for all words $u, v \in \Sigma^*$ which are permutations of each other. This is the unifying view that we shall adopt.

In the context of formal languages and automata theory, series in noncommuting indeterminates have been introduced by Schützenberger, who considered the class of *rational series* [Schutzenberger:IC:1961] (cf. [BerstelReutenauer:CUP:2010] for an extensive introduction). This class has since appeared in various areas of computer science and mathematics [HandbookWA], such as speech recognition [MohriPereira:Riley:CSL:2002], digital image compression [HandbookWA], learning [BalleMohri:AI:2015], quantum computing [BlondelJeandelKoiranPortier:SIAM:JoC:2005], as well as in Bell and Smertnig’s resolution [BellSmertnig:SM:2021] of a conjecture of Reutenauer [Reutenauer:FCT:1979].

Series have also been applied to control theory, as put forward by the pioneering work of Fliess [Fliess:1981]. Namely, he associated to a control system its *characteristic series*, in such a way that the behaviour of the system as an input-output transformer is uniquely determined by it [Fliess:1981]. He then recovered the rational series as the characteristic series of the so-called *bilinear systems* [Fliess:MST:1976]. This is an extensively studied class [DAlessandroIsidoriRuberti:SIAMJoC:1974], whose theory can be derived from well-known results for rational series (e.g., decidability of equivalence checking and existence of minimal realisations). It is in relation with control theory that commutativity naturally enters the picture. Fliess isolated an important subclass of bilinear systems, namely those where the output at time t depends only on t and the inputs at time t (but not at times $< t$), and proved that their characteristic series coincide with the *commutative* rational series [Fliess:MST:1976].

Commutativity, besides arguably being a fundamental algebraic notion per se, also appears in a number of other works. For instance, it is used as a simplifying assumption in Karhumäki’s study on $\mathbb{N}$ -rational series [Karhumaki:TCS:1977] (later corrected in [Lopez:STACS:2025]), it is featured in Litow’s short note on the undecidability of the multivariate Skolem problem [Litow:IPL:2003], in the study of DTOL systems [Karhumaki:TCS:1979], in quantum finite automata [HuangCao:CTISC:2021], in the formal verification of MapReduce frameworks [ChenSongWu:CAV:2016], and in program verification [Farzan:LICS:2023].

All these works leveraging commutativity offer no indication as whether one can decide it algorithmically. Moreover, most decision problems for rational series are undecidable e.g., *nonemptiness* and *containment* [NasuHonda:IC:1969] (cf. also [Paz:1971]), therefore finding problems which are decidable for the whole class is challenging.

Contributions

We study the *commutativity problem* for classes of series from a computational perspective. For rational series, thanks to strong structural properties [Fliess:1971-1972] commutativity reduces to whether the linear maps in a *minimal linear representation* commute pairwise. Thanks to Schützenberger’s seminal result, such minimal representations can be computed efficiently [Schutzenberger:IC:1961], and thus we can check commutativity in polynomial time by a direct numerical computation. This is in line with the fact that decidable problems on rational series often rely on minimisation in an essential way [BellSmertnig:LICS:2023, Kostolanyi:IC:2023, JeckerMazowieckiPurser:LICS:2024]. For more general classes where minimisation is not available, commutativity is an open problem.

First contribution. We show that commutativity is decidable for *effective prevarieties* of series, a generalisation of Reutenauer’s *varieties* [Reutenauer_1980]. *Prevarieties* are classes of series closed under linear combinations, and right and left derivatives (cf. § 2). *Effectiveness* demands that series in the class can be manipulated by algorithms working on finite representations. This achieves a delicate balance: It is sufficiently strong to ensure decidability of commutativity, but sufficiently weak to be applicable to several classes of series not known to have minimal representations.

Theorem 1. *Let S be an effective prevariety of series. The commutativity problem for S is decidable.*

The theorem relies on a finite axiomatisation of commutativity (Lemma 4) and provides a uniform framework in which many classes of series can be shown to have decidable commutativity. For example, rational series are effective (pre)varieties [Reutenauer_1980], giving an alternative decidability proof not relying on minimisation.

Second contribution. We apply Theorem 1 to decide commutativity for three, mutually incomparable classes of series, generalising the rational ones. Namely, we consider series recognised by *polynomial automata* [BenediktDuffSharadWorrell:PolyAut:LICS:2024], *shuffle automata* [Clemente:CONCUR:2024] (§ 6), and *infiltration automata* (§ 7). Polynomial automata (also known as *cost register automata* over the rationals [AlurDAntoniDeshmukhRaghothamanYuan:LICS:2013]) and shuffle automata are known to have a decidable equality problem [BenediktDuffSharadWorrell:PolyAut:LICS:2017, Clemente:CONCUR:2024], however they have very different computational properties, e.g., Ackermannian for the former while elementary for the latter. They were not known to be prevarieties. The notion of infiltration automata is novel, as well as decidability of their equality and commutativity problems. Being effective prevarieties, we can decide commutativity for these classes in a common framework, which we find remarkable. As far as we know, no other natural problem beyond equality was known to be decidable for these classes.

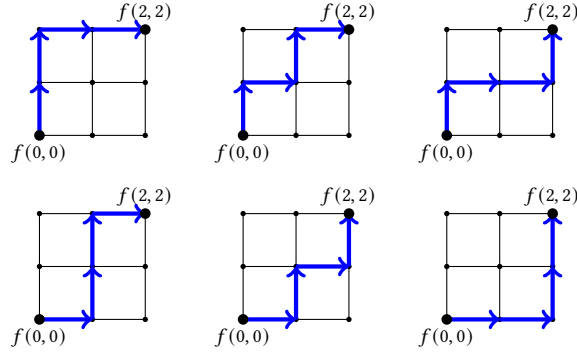
Theorem 2. *Polynomial automata, shuffle automata, and infiltration automata recognise effective prevarieties. As a consequence, the commutativity problem is decidable for them.*

In fact, for the classes above we show that equality reduces in polynomial time to commutativity, yielding a complete picture.

Example 1. As an application of Theorem 2, consider a polynomial automaton over alphabet $\Sigma = \{a_1, a_2\}$ with one register r . Initially, the register starts at some rational value $r := c \in \mathbb{Q}$. Upon reading a_1 , it is updated with the rule $r := r^2$ and upon reading a_2 with $r := 1 - r^2$. After reading the input, the automaton outputs the value of r . For instance, reading $a_1 a_2$, the automaton outputs $1 - c^4$, while over $a_2 a_1$ it outputs $(1 - c^2)^2$. For $c = -1$, these two outputs coincide, and in fact our algorithms determines that the semantics of the automaton is commutative for this initial value. For other values, e.g., $c = 2$, the semantics is not commutative: $a_1 a_2 \mapsto 1 - 2^4 = -15$, but $a_2 a_1 \mapsto (1 - 2^2)^2 = 9$.

Third contribution. We apply Theorem 2 to two problems on series in commuting variables. We find it intriguing that we can solve problems about series in commuting variables using an algorithmic result in the theory of noncommuting variables. This corroborates the hypothesis that the latter are a more fundamental object than the former.

Multivariate polyrec sequences. In the first problem, we propose a notion of *multivariate polynomial recursive sequences* (*polyrec*), which are a subset of $\mathbb{N}^d \rightarrow \mathbb{Q}$ generalising the univariate polyrec sequences from [Cadilhac:Mazowiecki:Paperman:Pilipczuk:2024]. Multivariate polyrec sequences are defined via systems of *polynomial difference equations* of a certain form (cf. § 5.4 for a precise definition). However, such systems may not have any solution at all, and it is not clear whether this can be decided. Solvability of polynomial difference equations is in general undecidable (Remark 9), which poses an obstacle to

Fig. 1. The six constraints on the computation of $f(2, 2)$.

the development of a computational theory of multivariate polyrec sequences. We overcome this difficulty by showing that commutativity of polynomial automata can be used to decide whether polyrec systems are solvable (Theorem 18). Thus, the syntax of multivariate polyrec sequences is effective.

Example 2. Consider the bivariate polyrec recursions

$$\begin{aligned} f(n_1 + 1, n_2) &= f(n_1, n_2)^2, \\ f(n_1, n_2 + 1) &= 1 - f(n_1, n_2)^2, \end{aligned}$$

for all $n_1, n_2 \in \mathbb{N}$. As shown in Figure 1, $f(2, 2)$ can be computed in $\binom{2+2}{2,2} = 6$ possible ways, all of which must give the same value. Polyrec equations can be interpreted as series constraints

$$\begin{aligned} f(a_1 \cdot w) &= f(w)^2, \\ f(a_2 \cdot w) &= 1 - f(w)^2, \end{aligned}$$

for all $w \in \Sigma^*$. These are essentially the same as the transitions of the polynomial automaton from Example 1. Such equations have a *unique series solution* $f : \Sigma^* \rightarrow \mathbb{Q}$ once we fix the initial value $f(\varepsilon) \in \mathbb{Q}$. Moreover, when this series solution is commutative, it will also satisfy the polyrec equations. And vice versa, when the polyrec equations have a solution, then this series is commutative. The 6 constraints in Figure 1 correspond to the commutativity conditions $f(a_1 a_1 a_2 a_2) = f(a_1 a_2 a_1 a_2) = f(a_1 a_2 a_2 a_1) = f(a_2 a_1 a_1 a_2) = f(a_2 a_1 a_2 a_1) = f(a_2 a_2 a_1 a_1)$. In this way, our algorithm determines the existence of a multivariate sequence solution to polyrec equations $f : \mathbb{N}^2 \rightarrow \mathbb{Q}$ starting at $f(0, 0) = -1$ by checking commutativity of the associated polynomial automaton from Example 1.

Multivariate CDA series. In the second problem, we consider the notion of *constructible differentially algebraic* (CDA) power series in commuting variables [BergeronReutenauer:EJC:1990]. They were introduced in the multivariate context by Bergeron and Sattler [BergeronSattler:TCS:1995], as solutions of systems of *polynomial differential equations* of a restricted form (cf. § 6.3 for details). Such systems may not have solutions and deciding whether this is the case is open [Clemente:CONCUR:2024]. Solvability of polynomial differential equations is undecidable by a result of Denef and Lipshitz [DenefLipshitz:1984] (Remark 11), and thus it is not clear whether CDA series have a

decidable syntax. We overcome this issue by showing that commutativity of shuffle automata can be used to decide whether CDA systems are solvable (Theorem 26), thus answering the open question positively.

Organisation

In the next section § 2 we present some preliminaries, followed in § 3 by a proof of Theorem 1. In § 4 we recall some known results about finite weighted automata, which will help to put the following results in perspective. Polynomial automata will be studied in § 5, followed by shuffle automata in § 6, and infiltration automata in § 7. (Thus the proof of Theorem 2 is spread over these three sections.) In the last section, an important remark shows that our techniques do not only solve the zeroness, equality, and commutativity problems, but in fact we can construct a finite representation for the *effective algebraic variety* of all initial conditions ensuring zeroness, equality, resp., commutativity (Remark 14).

This paper is an extended version of [Clemente:LICS:2025]. In comparison to [Clemente:LICS:2025], we have included more details and all interesting proofs. Moreover, we provide the complete development for infiltration automata (§ 7), which before was just sketched. We have also introduced a new closure property, namely closure under left anti-derivatives (§ 3), which allows us to provide a very simple argument showing that the equality problem reduces back to the commutativity problem. The preliminary section § 4 providing a gentle introduction in the linear case is also new.

2 Preliminaries

We say that a structure (such as a class of series, a class of languages, an algebra, etc...) is *effective* if its elements can be finitely represented (with a decidable syntax), equality between elements can be decided based on their representations, and all the structure operations can be carried over algorithmically. For instance the class of regular languages is effective, since they can be represented by finite automata (amongst many other options) and language equality is decidable. Most results in the paper hold for any field, however for computability considerations we need to restrict ourselves to effective fields. For concreteness, we consider the field of rational numbers $\mathbb{Q}$, however all our results hold for other effective fields, such as the real numbers $\mathbb{R}$ or the complex numbers $\mathbb{C}$.

Series. Let $\Sigma = \{a_1, \dots, a_d\}$ be a finite input alphabet. We denote by Σ^* the set of *finite words* over Σ , a monoid under the operation of concatenation, with neutral element the empty word ε . The main object of study in this paper are *series* over $\mathbb{Q}$, which are mappings $\Sigma^* \rightarrow \mathbb{Q}$. These are sometimes called *weighted languages* and are denoted by $\mathbb{Q}\langle\langle\Sigma\rangle\rangle$. We refer to [BerstelReutenauer:CUP:2010] for a general introduction. We use f, g, h to denote series in $\mathbb{Q}\langle\langle\Sigma\rangle\rangle$, and we write a series f as $\sum_{w \in \Sigma^*} f_w \cdot w$, where the value of f at w is $f_w \in \mathbb{Q}$. We also write $[w]f$ for f_w . The *support* of a series f is the set of words w such that $f_w \neq 0$. A *polynomial* is a series with finite support. Thus, $3aab - \frac{5}{2}bc$ is a polynomial and $1 + a + a^2 + \dots$ is a series but not a polynomial.

The set of series is equipped with a variety of operations. It carries the structure of a vector space, with zero 0 , *scalar multiplication* $c \cdot f$ with $c \in \mathbb{Q}$, and *addition* $f + g$ all defined element-wise by

$$[w]0 := 0, \quad [w](c \cdot f) := c \cdot [w]f, \quad \text{and} \quad [w](f + g) := [w]f + [w]g, \quad \forall w \in \Sigma^*.$$

This vector space is equipped with two important families of linear maps, called *left* and *right derivatives* $\delta_a^L, \delta_a^R : \mathbb{Q}\langle\langle\Sigma\rangle\rangle \rightarrow \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, for every $a \in \Sigma$. They are the series analogues of language quotients: For every series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$,

they are defined element-wise by

$$[w](\delta_a^L f) := [a \cdot w]f, \quad \text{resp.,} \quad [w](\delta_a^R f) := [w \cdot a]f, \quad \forall w \in \Sigma^*.$$

The following elementary observation will be very useful.

Lemma 3 (Left and right derivatives commute). *For every two letters $a, b \in \Sigma$, we have $\delta_a^L \circ \delta_b^R = \delta_b^R \circ \delta_a^L$.*

(By $\delta_a^L \circ \delta_b^R$ we mean the *composition* of the two derivatives, which is the function mapping a series f to the series $\delta_a^L \delta_b^R f$. We avoid writing parenthesis for function application when possible.) Derivatives are extended to all finite words homomorphically: $\delta_c^L f := f$ and $\delta_{a \cdot w}^L f := \delta_w^L(\delta_a^L f)$ for all $a \in \Sigma$ and $w \in \Sigma^*$ (note that the order is reversed); similarly for right derivatives.

Prevarieties of series. Let $\mathcal{S}$ be a class of series, and for an alphabet Σ let $\mathcal{S}_\Sigma$ the subset of $\mathcal{S}$ consisting of series over input alphabet Σ . We require that $\mathcal{S}_\Gamma \supseteq \mathcal{S}_\Sigma$ for every alphabet $\Gamma \supseteq \Sigma$. A class of series $\mathcal{S}$ is a *prevariety* if the following two conditions hold, for every alphabet Σ :

(V.1) $\mathcal{S}_\Sigma$ is a vector space over $\mathbb{Q}$.

(V.2) Closure under derivatives: For every series $f \in \mathcal{S}_\Sigma$ and letter $a \in \Sigma$, the series $\delta_a^L f$ and $\delta_a^R f$ are in $\mathcal{S}_\Sigma$.

(A *variety*, as introduced by Reutenauer, is a prevariety which satisfies an additional condition (cf. [Reutenauer_1980]); we will not need this notion in the rest of the paper.) The *equality problem* for a prevariety of series $\mathcal{S}$ is the following decision problem: Given (finite representations of) two series $f, g \in \mathcal{S}$, is it the case that $f = g$? In the special case when $g = 0$ we obtain the *zeroness problem*. A class of series $\mathcal{S}$ is an *effective prevariety* if it is a prevariety, and moreover

- (1) Every series in $\mathcal{S}$ admits a decidable finite presentation.
- (2) The closure properties (V.1) and (V.2) can be carried over algorithmically by manipulating finite presentations.
- (3) The equality problem is decidable for series in $\mathcal{S}$.

Thanks to the equivalence $f = g \Leftrightarrow f - g = 0$ and (V.1), the notion of effective prevariety does not change if we replace the third condition above with the following seemingly weaker one:

- 3') the zeroness problem is decidable for series in $\mathcal{S}$.

For instance, the class of rational series is an effective prevariety [Reutenauer_1980], as we recall later in section § 4.

3 Commutativity problem

We introduce the main problem that we study. The *commutative image* of a word $w \in \Sigma^*$ (sometimes called *Parikh image*) over a totally ordered alphabet $\Sigma = \{a_1, \dots, a_d\}$ is the vector $c(w) := (\#(w)_{a_1}, \dots, \#(w)_{a_d}) \in \mathbb{N}^d$, where $\#(w)_{a_j}$ is the number of occurrences of a_j in w . For two words u, v we write $u \sim v$ if they have the same commutative image, i.e., one word can be obtained from the other by permuting the positions of letters. For instance, $a_1 a_1 a_2 \sim a_1 a_2 a_1 \sim a_2 a_1 a_1$, since the commutative image of all three words is $(2, 1) \in \mathbb{N}^2$. A series $f \in \mathbb{Q}\langle\langle \Sigma \rangle\rangle$ is *commutative* (called *échangeable* in [Fliess:1981]) if for every two words $u, v \in \Sigma^*$ with the same commutative image $u \sim v$, the series produces the same value $f_u = f_v$.

On the face of it, commutativity demands to check infinitely many equalities. Our simple, but crucial observation is that it can be characterised by finitely many equations. Clearly, if f is commutative, then for every input symbols

$a, b \in \Sigma$ we have

$$\begin{aligned} \delta_a^L \delta_b^L f &= \delta_b^L \delta_a^L f, & \text{and} & & (\text{swap}) \\ \delta_a^L f &= \delta_a^R f. & & & (\text{rotate}) \end{aligned}$$

Indeed, the first equation corresponds to exchanging the first two input letters $abw \sim baw$, and the second equation corresponds to a rotation $aw \sim wa$. Since swaps and rotations suffice to generate all commutatively equivalent words, we obtain the following characterisation.

Lemma 4 (Characterisation of commutativity). *A series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is commutative if, and only if, it satisfies equations (swap) and (rotate), for every $a, b \in \Sigma$.*

Thanks to Lemma 4, commutativity is decidable for effective prevarieties of series. Indeed, if $\mathcal{S}$ is a prevariety and $f \in \mathcal{S}$, then $\delta_a^L f, \delta_a^R f \in \mathcal{S}$, as well as all the other series mentioned in (swap) and (rotate). By effectiveness, we can construct finite representations for these series and decide the equality tests. This observation will allow us to decide commutativity for rich classes of series for which the commutativity problem is nontrivial.

Theorem 1. *Let $\mathcal{S}$ be an effective prevariety of series. The commutativity problem for $\mathcal{S}$ is decidable.*

Remark 1. Theorem 1 allows us to reduce commutativity to finitely many equality tests. The observation that commutativity can be characterised by finitely many equations is certainly not novel. For instance, in the context of formal verification one could find an analogous statement in [ChenSongWu:CAV:2016].

The three classes of series that we consider in §§ 5 to 7 will turn out to be effective prevarieties, and thus commutativity is decidable for them. One may wonder whether equality reduces back to commutativity, so that the two problems are in fact inter-reducible. This will indeed be the case, thanks to an additional closure property that we introduce next. We say that a series $g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is a *left anti-derivative* of a tuple of series $f = (f_a)_{a \in \Sigma} \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle^\Sigma$ if $\delta_a^L g = f_a$ for all $a \in \Sigma$. Clearly, once we fix the initial condition $g_\varepsilon \in \mathbb{Q}$, anti-derivatives are unique. A class of series $\mathcal{S}$ is *closed under left anti-derivatives* if whenever g is a left anti-derivative of a tuple of series f from $\mathcal{S}_\Sigma$, then g is also in $\mathcal{S}_\Sigma$.

Lemma 5. *Let $\mathcal{S}$ be an effective prevariety of series, except that the equality problem is not assumed to be decidable. If $\mathcal{S}$ is effectively closed under left anti-derivatives, then the zeroness problem effectively reduces to the commutativity problem.*

PROOF. Consider a finite alphabet Σ and a series $f \in \mathcal{S}_\Sigma$ over Σ . Let $a, b \notin \Sigma$ be two fresh letters not in Σ and define the enlarged alphabet $\Gamma := \Sigma \cup \{a, b\}$. Let $g \in \mathcal{S}_\Gamma$ be the unique series such that $g_\varepsilon = g_x = 0$ for all $x \in \Gamma$, and, for every $x, y \in \Gamma$,

$$\delta_y^L \delta_x^L g = \begin{cases} f & \text{if } x = a, y = b, \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

Since $\mathcal{S}$ is closed under anti-derivatives, g is in $\mathcal{S}_\Gamma$. We claim that g is commutative if, and only if, $f = 0$. The “if” direction is clear since $f = 0$ implies $g = 0$, which is commutative. For the “only if” direction, assume that $f \neq 0$, so that there is a word $w \in \Sigma^*$ in its support $f(w) \neq 0$. Then $g(abw) = f(w) \neq 0$ however $g(baw) = 0$, and thus g is not commutative. $\square$

In §§ 5 to 7 we leverage Theorem 1 and Lemma 5 for three expressive classes of series, together with respective applications.

4 Linearly-finite series and recognisable series

This brief section does not contain any new results, however it will be instructive in comparison with the stronger results developed in §§ 5 to 7.

Linearly-finite series. We recall a semantic mechanism to define classes of series that will be useful in the rest of the paper in comparison to more general classes. Given a set of series $F \subseteq \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, let their *linear span*, denoted by $\text{Lin}_{\mathbb{Q}}(F)$, be the set of finite linear combinations of series from F , with coefficients from $\mathbb{Q}$. In other words, elements of this set are of the form $c_1 \cdot f_1 + \dots + c_n \cdot f_n$ for some $f_1, \dots, f_n \in F$ and $c_1, \dots, c_n \in \mathbb{Q}$. It is easy to see that $\text{Lin}_{\mathbb{Q}}(F)$ is a vector space over $\mathbb{Q}$ containing F , and in fact it is the smallest such vector space. Sets of series of the form $\text{Lin}_{\mathbb{Q}}(f_1, \dots, f_k)$ are called *finitely generated*, with *generators* $f_1, \dots, f_k$. We say that a series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is *linearly finite* if it belongs to a finitely generated $\mathbb{Q}$ -vector space of series closed under left derivatives δ_a^L , $a \in \Sigma$. In other words, f is linearly finite when there are generators $f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ s.t.:

(R.1) $f \in \text{Lin}_{\mathbb{Q}}(f_1, \dots, f_k)$ and

(R.2) $\delta_a^L f_i \in \text{Lin}_{\mathbb{Q}}(f_1, \dots, f_k)$ for every $a \in \Sigma$ and $1 \leq i \leq k$,

Remark 2. We would get the same class if we additionally required that the generators are obtained from f itself by a finite number of left derivatives. In other words, we could take the generators to be of the form $f_i = \delta_{w_i}^L f$ for some words $w_i \in \Sigma^*$. Since this will not be the case for the more general classes of series that we will consider in the rest of the paper, we have chosen the definition above.

Lemma 6. *The class of linearly finite series is effectively closed under scalar product, addition, left derivatives and anti-derivatives.*

PROOF. Closure under scalar product follows from the definition of linear span. Regarding closure under addition, let f, g be linearly finite series with generators F , resp., G . Then $f + g$ is linearly finite with generators $F \cup G$. Closure under left derivatives holds by design: If f is linearly finite with generators G , then for every $a \in \Sigma$ the series $\delta_a^L f$ is also linearly finite with the same set of generators. Finally, for closure under anti-derivatives consider linearly finite series $f_1, \dots, f_d$ with generators, resp., $G_1, \dots, G_d$ s.t. $\delta_{a_j}^L f = f_j$ for every $1 \leq j \leq d$. Then f is linearly finite with generators $\{f, f_1, \dots, f_d\} \cup G_1 \cup \dots \cup G_d$. $\square$

The class of linearly finite series is also closed under right derivatives, however we will obtain this from a stronger closure property, namely closure under reversal. To show the latter, we will use a stronger representation for linearly finite series, which we recall next.

Linear representations. A *linear representation* is a tuple (Σ, k, x, y, M) , where $\Sigma = \{a_1, \dots, a_d\}$ is a finite alphabet, $k \in \mathbb{N}$ is the *dimension* of the representation, $x \in \mathbb{Q}^{1 \times k}$ is a row vector of *initial weights*, $y \in \mathbb{Q}^{k \times 1}$ is a column vector of *final weights*, and $M : \Sigma \rightarrow \mathbb{Q}^{k \times k}$ is the *transition function*, assigning to each input letter $a \in \Sigma$ a transition matrix $M(a) \in \mathbb{Q}^{k \times k}$. The transition function M is extended homomorphically to all input words $M : \Sigma^* \rightarrow \mathbb{Q}^{k \times k}$ by $M(\varepsilon) := I_k$ (the $k \times k$ identity matrix) and $M(a \cdot w) := M(a) \cdot M(w)$ for every $a \in \Sigma$ and $w \in \Sigma^*$. A linear representation *recognises* the series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ mapping an input word $w \in \Sigma^*$ to $[w]f := x \cdot M(w) \cdot y$. A series is *recognisable* if it is recognised by some linear representation. The following coincidence result is classical (cf. [BerstelReutenauer:CUP:2010]).

Lemma 7. *A series is recognisable if, and only if, it is linearly finite.*

Thanks to the lemma above, recognisable series are closed under the operations from Lemma 6. The advantage of linear representations is that they make it possible to show stronger closure properties, as we now demonstrate. The *reversal* of series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is the series $f^{\text{rev}} \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ which reverses the order of the letters in the input. That is, for every input word $w \in \Sigma^*$ we have $[w](f^{\text{rev}}) := [w^{\text{rev}}]f$, where w^{rev} is the word obtained from w by reversing the order of its letters. For instance, $(abc)^{\text{rev}} = cba$. Note that reversal is an endomorphism on the vector space of series.

Lemma 8. *If $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is recognisable, then f^{rev} is also recognisable.*

PROOF. Let f be recognised by a linear representation (Σ, k, x, y, M) . Then its reversal f^{rev} is recognised by the linear representation $(\Sigma, k, y^T, x^T, M^T)$, where M^T is the transpose of M , and similarly for y^T, x^T . Indeed, for every input word $w \in \Sigma^*$ we have $[w](f^{\text{rev}}) = [w^{\text{rev}}]f = x \cdot M(w^{\text{rev}}) \cdot y = x \cdot (M^T(w))^T \cdot y = y^T \cdot M^T(w) \cdot x^T$. $\square$

Notice that it is not so clear how to prove closure under reversal solely based on the definition of linearly finite series. Since recognisable series are closed under left derivatives and reversal, by double reversal $\delta_a^R f = (\delta_a^L f^{\text{rev}})^{\text{rev}}$ we obtain closure under right derivatives as well.

Corollary 9. *If f is recognisable then $\delta_a^R f$ is also recognisable.*

Theorem 10. *The class of recognisable (= linearly finite) series is an effective prevariety. In particular, the commutativity problem is decidable.*

Towards more general classes. In the next sections §§ 5 to 7 we will consider classes of series which generalise the recognisable ones. In each case, the more general class will be obtained by considering a notion of product of series, thus moving from a vector space structure to a more general algebra structure. The basic closure properties as in Lemma 6 will follow from first principles. Decidability of the zeroness problem will be obtained as an application of Hilbert finite basis theorem for finitely generated polynomial algebras with finitely many variables. The price to pay is that such classes of series will not be closed under reversal in general. Consequently, in order to show that they are prevarieties we will need to show closure under right derivatives by other means.

5 Hadamard automata and Hadamard-finite series

In this section, we study a weighted model of computation strongly related to polynomial automata and the class of series it recognises. We begin in § 5.1 with some mathematical preliminaries. In § 5.2 we define Hadamard-finite series, generalising the recognisable series, and present their closure properties. Hadamard automata are defined and studied in § 5.3, where they are shown to recognise precisely the Hadamard-finite series. This leads to the main result of the section: Hadamard-finite series constitute an effective prevariety (Theorem 16) and thus the commutativity problem is decidable for this model (Theorem 17). We conclude in § 5.4 with an application of the commutativity problem to a class of multivariate number sequences defined by polynomial recursions (Theorem 18).

5.1 Preliminaries

5.1.1 Difference algebra. Let $\mathbb{A} = (A; 0, 1, +, *)$ be an *algebra* (over the field of rational numbers), that is a vector space equipped with an associative commutative bilinear product with identity 1 [Bourbaki:Algebra:I:1974]. An algebra *endomorphism* is a linear function $F : A \rightarrow A$ s.t.

$$F(1) = 1 \quad \text{and} \quad F(\alpha * \beta) = F\alpha * F\beta, \quad \text{for all } \alpha, \beta \in A. \quad (\text{endomorphism rule})$$

A *difference algebra* [RMCohn:DifferenceAlgebra:1965, Levin:DifferenceAlgebra:2008] is an algebra $(A; 0, 1, +, *, (F_j)_{1 \leq j \leq d})$ together with finitely many endomorphisms $F_1, \dots, F_d : A \rightarrow A$. Unlike what is usually assumed in the difference algebra literature, we do not require the F_i 's to pairwise commute. Difference algebras will provide the semantic background for the rest of the section.

As an example, consider the algebra of multivariate polynomials $(\mathbb{Q}[X]; 0, 1, +, \cdot)$, where $X = (X_1, \dots, X_k)$ is a tuple of commuting variables. An endomorphism F acts on a polynomial $\alpha \in \mathbb{Q}[X]$ just by evaluation $F\alpha = \alpha(F(X_1), \dots, F(X_k))$, and thus it is uniquely defined once we have fixed $F(X_1), \dots, F(X_k) \in \mathbb{Q}[X]$. If we fix distinguished endomorphisms $F_1, \dots, F_d$, then the algebra of polynomials acquires the structure of a difference algebra $(\mathbb{Q}[X]; 0, 1, +, \cdot, (F_j)_{1 \leq j \leq d})$. Other examples of difference algebras will be presented in §§ 5.1.2 and 5.4.1.

5.1.2 Hadamard difference algebra. We define a product operation on series which will be central for the development of this section. The *Hadamard product* [Fliess:1974] of two series $f, g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, denoted by $f \odot g$, is defined element-wise: For every $w \in \Sigma^*$, $[w](f \odot g) := [w]f \cdot [w]g$. For instance, if we have two polynomials $2 \cdot aab - c$ and $3 \cdot aab + b$ (in noncommuting variables), their Hadamard product is $6 \cdot aab$. This is an associative and commutative operation, it distributes over sum, and the identity element is the series $\mathbb{1} = \sum_{w \in \Sigma^*} 1 \cdot w$ mapping every word to 1, giving rise to the *Hadamard algebra* $(\mathbb{Q}\langle\langle\Sigma\rangle\rangle; \mathbb{0}, \mathbb{1}, +, \odot)$. The Hadamard product for series is reminiscent of the notion of *intersection* in language theory and of *fully synchronous composition* in concurrency theory.

In order to make it easier to compare the Hadamard product to the shuffle and infiltration products studied later, we find it convenient to provide the following alternative, coinductive definition (cf. [BasoldHansenPinRutten:MSCS:2017]): For any two series $f, g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, their Hadamard product $f \odot g$ is the unique series s.t.

$$[\varepsilon](f \odot g) = f_\varepsilon \cdot g_\varepsilon, \quad (\odot-\varepsilon)$$

$$\delta_a^L(f \odot g) = \delta_a^L f \odot \delta_a^L g, \quad \forall a \in \Sigma. \quad (\odot-\delta_a^L)$$

Equation $(\odot-\delta_a^L)$ states that left derivatives are endomorphisms w.r.t. the Hadamard product. We thus obtain the *Hadamard difference algebra* of series $\mathbb{Q}\langle\langle\Sigma\rangle\rangle_\odot := (\mathbb{Q}\langle\langle\Sigma\rangle\rangle; \mathbb{0}, \mathbb{1}, +, \odot, (\delta_a^L)_{a \in \Sigma})$, which will provide the semantic background for the forthcoming Hadamard automata. Right derivatives are also Hadamard algebra endomorphisms, which will be useful later.

Proposition 11. *Right derivatives δ_a^R (with $a \in \Sigma$) are Hadamard algebra endomorphisms. I.e., they are linear, preserve the multiplicative identity $\delta_a^R \mathbb{1} = \mathbb{1}$, and commute with Hadamard product:*

$$\delta_a^R(f \odot g) = \delta_a^R f \odot \delta_a^R g, \quad \forall a \in \Sigma. \quad (\odot-\delta_a^R)$$

5.2 Hadamard-finite series

In this section we rely on the structure of the Hadamard algebra to define a finitely presented class of series. For series $f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, let $\mathbb{Q}[f_1, \dots, f_k]_\odot$ be the smallest Hadamard algebra containing $f_1, \dots, f_k$. Hadamard algebras of this form are called *finitely generated*, with *generators* $f_1, \dots, f_k$. It is a *difference Hadamard algebra* if it is additionally closed under the endomorphisms δ_a^L , $a \in \Sigma$.

Definition 1 (Hadamard-finite series). A series is *Hadamard finite* if it belongs to a finitely generated difference Hadamard algebra.

The following lemma will be our working definition of Hadamard-finite series.

Lemma 12. *A series f is Hadamard finite iff there are generators $f_1, \dots, f_k$ s.t.:*

- (1) $f \in \mathbb{Q}[f_1, \dots, f_k]_{\odot}$.
 - (2) $\delta_a^L f_i \in \mathbb{Q}[f_1, \dots, f_k]_{\odot}$ for every $a \in \Sigma$ and $1 \leq i \leq k$. I.e., there are polynomials $p_i^{(a)} \in \mathbb{Q}[y_1, \dots, y_k]$ s.t.
- $$\delta_a^L f_i = p_i^{(a)}(f_1, \dots, f_k), \quad \text{for all } a \in \Sigma, 1 \leq i \leq k. \quad (1)$$

Moreover, we can assume without loss of generality that f equals one of the generators $f = f_1$.

We call (1) a *system of Hadamard-finite equations*. Note that every choice for the constant terms $[\varepsilon]f_i \in \mathbb{Q}$ extends to a unique series solution of (1). Consequently, a Hadamard-finite series can be finitely presented by the constant terms and the polynomials $p_i^{(a)} \in \mathbb{Q}[y_1, \dots, y_k]$.

PROOF. If f is Hadamard finite, then the two conditions hold by definition. W.l.o.g. we can take $f = f_1$ to be one of the generators since we can just add f to the generators, without changing the validity of the two conditions.

For the other direction, assume that there are generators $f_1, \dots, f_k$ with $f = f_1$ satisfying the two conditions. We need to show that the finitely generated algebra $A := \mathbb{Q}[f_1, \dots, f_k]_{\odot}$ is closed under left derivatives. To this end, consider a series $g \in A$. There is a polynomial $p \in \mathbb{Q}[y_1, \dots, y_k]$ s.t. $g = p(f_1, \dots, f_k)$. Take now the left derivative of both sides. Since they are homomorphisms of A , we have $\delta_a^L g = \delta_a^L(p(f_1, \dots, f_k)) = p(\delta_a^L f_1, \dots, \delta_a^L f_k)$. By the second condition, $\delta_a^L f_1, \dots, \delta_a^L f_k \in A$, and thus $\delta_a^L g \in A$ since A is an algebra, as required. $\square$

Remark 3. Hadamard-finite equations have been considered under the name of *polynomial recurrent relations* by Sénizergues [Senizergues:CSR:2007], where they are claimed, without proof, to coincide with those recognised by deterministic pushdown automata of level 3 [Senizergues:CSR:2007], and to have a decidable equality problem [Senizergues:CSR:2007]. In the context of arbitrary semirings, they also appear under the name of *Hadamard-polynomial equations* [KostolanyiMisun:TCS:2018].

Remark 4 (Cauchy product and Cauchy-finite series). The definition of Hadamard-finite series (as well as the shuffle and infiltration-finite series from the next sections) follows a natural pattern. In fact, every algebra of series $(\mathbb{Q}\langle\langle\Sigma\rangle\rangle; 0, 1, +, *)$ gives rise to a corresponding class of **-finite* series: A series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is **-finite* if it belongs to a finitely generated sub-algebra closed under δ_a^L ($a \in \Sigma$). For instance, taking “*” to be the *Cauchy product* (also known as *concatenation product*) we obtain the *Cauchy-finite series*, which have been studied in [Wechler:MFCs:1979] and shown to coincide with the class of algebraic series. Note that Cauchy product is not commutative, unlike the products that we consider in this paper, and thus does not fall under the scope of the techniques that we develop.

Remark 5. When all the polynomials involved in the definition of a Hadamard-finite series have degree ≤ 1 (i.e., they are linear), we recover the notion of recognisable series from § 4.

We recall some basic closure properties of Hadamard-finite series. All of them follow straightforwardly from the definitions.

Lemma 13 (Closure properties of Hadamard-finite series). *The class of Hadamard-finite series is an effective difference algebra, i.e., it contains $0, 1$, and it is effectively closed under scalar product, sum, Hadamard product, and left derivatives. Moreover, Hadamard-finite series are effectively closed under right derivatives and left anti-derivatives.*

When we say that Hadamard-finite series are effectively closed under right derivatives we mean that there is an algorithm that, given in input a finite presentation for f and an input symbol $a \in \Sigma$, produces in output a finite

presentation for $\delta_a^R f$. A similar remark applies to the effectiveness of the other closure properties. Closure under anti-derivatives will be used in conjunction to Lemma 5 to show that the commutativity problem is at least as hard as the equality problem.

PROOF. The series $\mathbb{0}$ and $\mathbb{1}$ are obviously Hadamard finite. Effective closure under scalar product $c \cdot f$ (with $c \in \mathbb{Q}$), addition $f + g$, left derivatives $\delta_a^L f$, and left anti-derivatives is done as in Lemma 6. Regarding closure under Hadamard product $f \odot g$, it is easy to show that if $f_1, \dots, f_k$ and $g_1, \dots, g_\ell$ are generators for f , resp., g , then $f_1, \dots, f_k, g_1, \dots, g_\ell$ can be taken to generate $f \odot g$. We consider now closure under right derivatives. Consider a Hadamard-finite series $f \in A$ for the finitely generated Hadamard algebra

$$A := \mathbb{Q}[f_1, \dots, f_k]_\odot$$

closed under left derivatives. Fix an input symbol $a \in \Sigma$. We consider new generators $\delta_a^R f_i$, for every $1 \leq i \leq k$, yielding the finitely generated Hadamard algebra

$$B := \mathbb{Q}[\delta_a^R f_1, \dots, \delta_a^R f_k]_\odot.$$

By Lemma 12, the proof is concluded by the following two claims.

Claim. $\delta_a^R f \in A$.

PROOF OF THE CLAIM. Since f is in the Hadamard algebra generated by the f_i 's and right derivative is an endomorphism by Proposition 11, $\delta_a^R f$ is in the Hadamard algebra generated by the $\delta_a^R f_i$'s, i.e., $\delta_a^R f \in A$. $\square$

Claim. For every $b \in \Sigma$ and $1 \leq i \leq k$, $\delta_b^L \delta_a^R f_i \in A$.

PROOF OF THE CLAIM. Since left and right derivatives commute by Lemma 3, we have $\delta_b^L \delta_a^R f_i = \delta_a^R \delta_b^L f_i$. But $\delta_b^L f_i$ is in the Hadamard algebra generated by the f_i 's and since right derivative is an endomorphism by Proposition 11, $\delta_a^R \delta_b^L f_i \in A$. $\square$

In this section we have defined the class of Hadamard-finite series in a semantic fashion, based on which we have shown effective closure properties in Lemma 13. In order to show that Hadamard-finite series constitute an effective prevariety, it remains to show that the zeroness problem is decidable for them. In order to achieve this, we introduce in the next section a syntactic presentation of the same class via a corresponding notion of automata.

5.3 Hadamard automata

We define the central computation model of the section.

Syntax. A *Hadamard automaton* is a tuple $\mathcal{A} = (\Sigma, X, F, \Delta)$ where Σ is a finite *input alphabet*, $X = \{X_1, \dots, X_k\}$ is a finite set of *nonterminal symbols*, $F : X \rightarrow \mathbb{Q}$ is the *output function* (or *initial condition*), and $\Delta : \Sigma \times X \rightarrow \mathbb{Q}[X]$ is the *transition function*. A Hadamard automaton is *linear* if $\Delta_a X_i$ is a polynomial of degree ≤ 1 for all $a \in \Sigma$ and $1 \leq i \leq k$.

Semantics. A *configuration* of a Hadamard automaton is a polynomial $\alpha \in \mathbb{Q}[X]$. For every input symbol $a \in \Sigma$, the transition function $\Delta_a : X \rightarrow \mathbb{Q}[X]$ is extended to a unique endomorphism $\Delta_a : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ of the polynomial algebra of configurations. In other words, Δ_a acts by evaluation: $\Delta_a(\alpha) = \alpha(\Delta_a(X_1), \dots, \Delta_a(X_k))$. Finally, transitions from single letters are extended to all finite input words homomorphically: For every configuration $\alpha \in \mathbb{Q}[X]$, input

word $w \in \Sigma^*$, and letter $a \in \Sigma$, we have $\Delta_\epsilon \alpha := \alpha$ and $\Delta_{a \cdot w} \alpha := \Delta_w \Delta_a \alpha$. We have all ingredients to define the *semantics* of a Hadamard automaton $\mathcal{A}$, which is a function $\mathcal{A}[\![_]\!]: \mathbb{Q}[X] \rightarrow \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ mapping a configuration $\alpha \in \mathbb{Q}[X]$ to the series $\mathcal{A}[\![\alpha]\!] \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ s.t.

$$[w](\mathcal{A}[\![\alpha]\!]) := F\Delta_w \alpha, \quad \text{for all } w \in \Sigma^*. \quad (2)$$

In other words, from the initial configuration α the automaton reads w and goes to configuration $\Delta_w \alpha$, from which the output function F is applied to produce the output value $F\Delta_w \alpha \in \mathbb{Q}$. For this last step, F is extended from nonterminals to configurations homomorphically $F\beta = \beta(FX_1, \dots, FX_k)$. The series *recognised* by the automaton $\mathcal{A}$ is $\mathcal{A}[\![X_1]\!]$.

Remark 6. The class of series recognised by linear Hadamard automata coincide with Schützenberger’s recognisable series [Schützenberger:IC:1961], which have been discussed in § 4.

Example 3. Revisiting Example 1 in the syntax of Hadamard automata, we have a single nonterminal $X = \{A\}$, output function F_c defined by $F_c A := c$ ($c \in \mathbb{Q}$), and transitions by

$$\Delta_{a_1} A := A^2 \quad \text{and} \quad \Delta_{a_2} A := 1 - A^2.$$

For instance, when $c = 0$ starting from the initial configuration A and reading input $u = a_1 a_2$ we obtain configuration $\Delta_{a_2} \Delta_{a_1} A = \Delta_{a_2} A^2 = (1 - A^2)^2$ and thus $\mathcal{A}[\![A]\!]_u = 1$. Reading $v = a_2 a_1$ yields a different configuration $\Delta_{a_1} \Delta_{a_2} A = 1 - A^4$, however $\mathcal{A}[\![A]\!]_v = 1$. In fact, $\mathcal{A}[\![A]\!]_w$ is the count modulo 2 of the number of a_2 ’s in w , and thus $\mathcal{A}[\![A]\!]$ is a commutative series.

It can be checked that also the output functions F_1, F_{-1} defined by $F_1 A := 1$, resp., $F_{-1} A := -1$ give rise to commutative series. However, no other choice does. For instance, F_2 defined by $F_2 A := 2$ does not yield a commutative series, since $\mathcal{A}[\![A]\!]_{a_1 a_2} = (1 - 2^2)^2 = 9$, but $\mathcal{A}[\![A]\!]_{a_2 a_1} = 1 - 2^4 = -15$.

A Hadamard automaton endows the algebra of configurations with the structure of a difference algebra of polynomials $(\mathbb{Q}[X]; 0, 1, +, \cdot, (\Delta_a)_{a \in \Sigma})$. This allows us to connect the difference algebra of configurations with the difference algebra of Hadamard series they recognise.

Lemma 14 (Properties of the semantics). *The semantics of a Hadamard automaton is a homomorphism from the difference algebra of polynomials $(\mathbb{Q}[X]; 0, 1, +, \cdot, (\Delta_a)_{a \in \Sigma})$ to the difference Hadamard algebra of series $(\mathbb{Q}\langle\langle\Sigma\rangle\rangle; 0, 1, +, \odot, (\delta_a^L)_{a \in \Sigma})$. In other words, $\mathcal{A}[\![0]\!] = 0$, $\mathcal{A}[\![1]\!] = 1$, and, for all $\alpha, \beta \in \mathbb{Q}[X]$,*

$$\mathcal{A}[\![c \cdot \alpha]\!] = c \cdot \mathcal{A}[\![\alpha]\!] \quad \forall c \in \mathbb{Q}, \quad (3)$$

$$\mathcal{A}[\![\alpha + \beta]\!] = \mathcal{A}[\![\alpha]\!] + \mathcal{A}[\![\beta]\!], \quad (4)$$

$$\mathcal{A}[\![\alpha \cdot \beta]\!] = \mathcal{A}[\![\alpha]\!] \odot \mathcal{A}[\![\beta]\!], \quad (5)$$

$$\mathcal{A}[\![\Delta_a \alpha]\!] = \delta_a^L(\mathcal{A}[\![\alpha]\!]) \quad \forall a \in \Sigma. \quad (6)$$

PROOF. Property $\mathcal{A}[\![0]\!] = 0$ is trivial, and $\mathcal{A}[\![1]\!] = 1$ holds by definition: $[w]\mathcal{A}[\![1]\!] = F\Delta_w 1 = F1 = 1$. Property (6) holds by definition of the semantics. Properties (3) and (4) follow from linearity of F and Δ_w . Finally, for property (5), we argue coinductively (the use of the coinductive assumption is marked by “!”):

$$[\epsilon](\mathcal{A}[\![\alpha \cdot \beta]\!]) = F(\alpha \cdot \beta) = F\alpha \cdot F\beta = [\epsilon](\mathcal{A}[\![\alpha]\!]) \cdot [\epsilon](\mathcal{A}[\![\beta]\!]) = [\epsilon](\mathcal{A}[\![\alpha]\!] \odot \mathcal{A}[\![\beta]\!]), \text{ and}$$

$$\delta_a^L(\mathcal{A}[\![\alpha \cdot \beta]\!]) = \mathcal{A}[\![\Delta_a(\alpha \cdot \beta)]\!] = \mathcal{A}[\![\Delta_a \alpha \cdot \Delta_a \beta]\!] \stackrel{!}{=} \mathcal{A}[\![\Delta_a \alpha]\!] \odot \mathcal{A}[\![\Delta_a \beta]\!] = \delta_a^L \mathcal{A}[\![\alpha]\!] \odot \delta_a^L \mathcal{A}[\![\beta]\!] = \delta_a^L(\mathcal{A}[\![\alpha]\!] \odot \mathcal{A}[\![\beta]\!]). \quad \square$$

Remark 7 (Hadamard vs. polynomial automata). Hadamard automata are closely related to *polynomial automata* [BenediktDuffSharadWorrell:PolyA], a common generalisation of weighted automata [Schutzenberger:IC:1961] and vector addition systems [Mayr:STOC:1981]. A state in a polynomial automaton is a tuple of rational numbers, which upon reading an input letter is modified by applying a polynomial update function; at the end of the run a polynomial output function is applied to produce the final weight. To compare, Hadamard automata start from an initial polynomial and modify it by applying polynomial endomorphisms; at the end of the computation the state polynomial is evaluated on a particular tuple of rational numbers. The two models are equivalent, up to reversal of the input: A series f is recognised by a Hadamard automaton iff its reversal f^R is recognised by a polynomial automaton. We show this equivalence in detail in § A.

Zeroneess for polynomial automata is decidable with Ackermann complexity [BenediktDuffSharadWorrell:PolyAut:LICS:2017], by an algorithm based on Hilbert’s finite basis theorem originating in the work of Novikov and Yakovenko [NovikovYakovenko:1999]. Since zeroness is invariant under reversal, this result also applies to Hadamard automata. Since commutativity is invariant under reversal, our results also apply to polynomial automata.

In the special case of an input alphabet with just one letter $\Sigma = \{a\}$, Hadamard (and polynomial) automata specialise to *polynomial recursive sequences* [Cadilhac:Mazowiecki:Paperman:Pilipczuk:Senizergues:ToCS:2021]. They have also been considered in [BorealeGorla:CONCUR:2021] in a coalgebraic setting, where zeroness is shown to be decidable by the same algorithm as for polynomial automata above specialised to $\Sigma = \{a\}$ [BorealeGorla:CONCUR:2021], however without complexity analysis. It is currently an open problem to find an algorithm deciding the equality problem for polynomial recursive sequences of elementary complexity [ClementeDontenBuryMazowieckiPilipczuk:STACS:23].

Over more general commutative semirings, polynomial automata have been introduced as *alternating weighted automata* [KostolanyiMisun:TCS:2018], further studied in [Grabolle:EPTCS:2021] (albeit without algorithmic results).

The following lemma states that Hadamard automata recognise precisely the Hadamard-finite series. Its proof relies on Lemmas 12 and 14.

Lemma 15. *A series is recognised by a Hadamard automaton if, and only if, it is Hadamard finite.*

PROOF. For the “only if” direction, let $f = \mathcal{A}[\![X_1]\!]$ be recognised by a Hadamard automaton $\mathcal{A} = (\Sigma, X, F, \Delta)$ with nonterminals $X = (X_1, \dots, X_k)$. We show that f is Hadamard finite by applying Lemma 12. Consider the Hadamard algebra $A := \mathbb{Q}[f_1, \dots, f_k]_{\odot}$ finitely generated by series $f_i := \mathcal{A}[\![X_i]\!]$ for all $1 \leq i \leq k$. Clearly $f \in A$. Now consider generator f_i and input symbol $a \in \Sigma$, and we need to show $\delta_a^L f_i \in A$. By Lemma 14, the semantics is a homomorphism of difference algebras, and thus $\delta_a^L f_i = \delta_a^L(\mathcal{A}[\![X_i]\!]) = \mathcal{A}[\![\Delta_a X_i]\!]$ $= (\Delta_a X_i)(\mathcal{A}[\![X_1]\!], \dots, \mathcal{A}[\![X_k]\!]) = (\Delta_a X_i)(f_1, \dots, f_k) \in A$, as required.

For the “if” direction, let $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ be Hadamard finite. By Lemma 12, there are series $f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ with $f = f_1$ generating a Hadamard algebra $A = \mathbb{Q}[f_1, \dots, f_k]_{\odot}$ s.t., for every generator f_i and input symbol $a \in \Sigma$, there is a polynomial $p_i^a \in \mathbb{Q}[X_1, \dots, X_k]$ s.t. $\delta_a^L f_i = p_i^a(f_1, \dots, f_k)$. We have all ingredients to build a Hadamard automaton recognising f_1 . Consider the automaton $\mathcal{A} = (\Sigma, X, F, \Delta)$ with nonterminals $X := (X_1, \dots, X_k)$, output mapping $F(X_i) := [\varepsilon]f_i$ and transitions

$$\Delta_a(X_i) := p_i^a(X_1, \dots, X_k), \quad \text{for all } 1 \leq i \leq k \text{ and } a \in \Sigma.$$

The proof is concluded by showing $\mathcal{A}[\llbracket X_i \rrbracket] = f_i$ for every $1 \leq i \leq k$. We argue coinductively. First, the two series have the same constant term by construction, $[\varepsilon](\mathcal{A}[\llbracket X_i \rrbracket]) = F\Delta_\varepsilon X_i = FX_i = [\varepsilon]f_i$. Second, the tails agree

$$\begin{aligned}
 \delta_a^L(\mathcal{A}[\llbracket X_i \rrbracket]) &= \mathcal{A}[\llbracket \Delta_a X_i \rrbracket] = && \text{(by Lemma 14)} \\
 &= \mathcal{A}[\llbracket p_i^a(X_1, \dots, X_k) \rrbracket] = && \text{(def. of } \Delta_a) \\
 &= p_i^a(\mathcal{A}[\llbracket X_1 \rrbracket], \dots, \mathcal{A}[\llbracket X_k \rrbracket]) = && \text{(by Lemma 14)} \\
 &= p_i^a(f_1, \dots, f_k) = && \text{(by coinduction)} \\
 &= \delta_a^L f_i. && \text{(def. of } p_i^a) \quad \square
 \end{aligned}$$

Example 4. We illustrate Lemma 15 on the Hadamard automaton from Example 3. Let $f := \mathcal{A}[\llbracket A \rrbracket]$. The corresponding Hadamard-finite equations are $\delta_{a_1}^L f = f^2$ and $\delta_{a_2}^L f = \mathbb{1} - f^2$. (The square operation is to be understood as Hadamard square, i.e., $f^2 = f \odot f$; recall that $\mathbb{1} = \sum_{w \in \Sigma^*} 1 \cdot w$ is the Hadamard identity.) It follows that $\mathbb{Q}[f]_\odot$ is a difference Hadamard algebra, and thus f is Hadamard finite.

Theorem 16. *The class of Hadamard-finite series is an effective prevariety.*

PROOF. The two closure conditions (V.1) and (V.2) follow from Lemma 13. By Lemma 15 and Remark 7, the equality problem for Hadamard-finite series is (effectively) equivalent to the equality problem for polynomial automata, and the latter is decidable by [BenediktDuffSharadWorrell:PolyAut:LICS:2017]. $\square$

We now have all ingredients to prove our main result for Hadamard-finite series.

Theorem 17. *The commutativity problem for Hadamard-finite series (equivalently, Hadamard and polynomial automata) is Ackermann-complete.*

PROOF. Thanks to Theorem 1 and Theorem 16, the commutativity problem for Hadamard-finite series is decidable. In fact, we have reduced the commutativity problem over alphabet Σ to $|\Sigma|^2 + |\Sigma|$ equivalence queries (cf. Lemma 4). On the other hand, by Lemma 13, Hadamard-finite series are effectively closed under anti-derivatives, and thus by Lemma 5 the equivalence problem for Hadamard-finite series reduces to the commutativity problem. Moreover, this reduction can be done efficiently, i.e., in polynomial time. Since the complexity of equivalence for polynomial automata is Ackermann-complete [BenediktDuffSharadWorrell:PolyAut:LICS:2017], the same is true for commutativity of Hadamard-finite series, Hadamard and polynomial automata. $\square$

5.4 Application: Multivariate polynomial recursive sequences

We introduce a class of multivariate recursive sequences generalising the univariate *polynomial recursive sequences* (in short, *polyrec*) from [Cadilhac:Mazowiecki:Paperman:Pilipczuk:Senizergues:ToCS:2021]. We will make a crucial use of Theorem 17 to argue that this class can be presented *effectively* by polynomial recursive equations. In other words, we will show that polynomial recursive equations are a *decidable syntax* for polyrec sequences: There is an algorithm which, given in input a system of polynomial recursive equations and an initial condition, decides whether they actually define a (polyrec) sequence at all. The lack of an effective presentation has been an obstacle to the study of a multivariate generalisation of polyrec sequences.

In § 5.4.1 we begin with some preliminaries about multivariate sequences, in § 5.4.2 we define multivariate polyrec sequences, whose basic properties we study in § 5.4.3. Finally in § 5.4.4 we use Theorem 17 to argue that their syntax is effective.

5.4.1 *Difference sequence algebra.* Fix a dimension $d \in \mathbb{N}_{\geq 1}$. For every $1 \leq j \leq d$, by e_j we denote the j -th unit vector in $\mathbb{N}^d$. A *multivariate sequence* is a function $f : \mathbb{N}^d \rightarrow \mathbb{Q}$. We equip the set of sequences with the structure of a difference algebra $(\mathbb{N}^d \rightarrow \mathbb{Q}; 0, 1, +, \cdot, (\sigma_j)_{1 \leq j \leq d})$, where $0, 1$, scalar product $c \cdot f$ ($c \in \mathbb{Q}$), sum $f + g$, and Hadamard product $f \cdot g$ are defined element-wise. Note that we overload the notation for the scalar and Hadamard product. Moreover, the j -th *left shift* endomorphism σ_j maps a sequence $f : \mathbb{N}^d \rightarrow \mathbb{Q}$ to the sequence $\sigma_j f$ s.t.

$$(\sigma_j f)(n) := f(n + e_j), \quad \text{for all } n \in \mathbb{N}^d.$$

Unlike the difference algebra of Hadamard series from § 5.1.2, the endomorphisms σ_j 's pairwise commute $\sigma_j \circ \sigma_h = \sigma_h \circ \sigma_j$.

5.4.2 *Polyrec sequences.* A d -variate sequence $f : \mathbb{N}^d \rightarrow \mathbb{Q}$ is *polyrec* if there exist $k \in \mathbb{N}_{\geq 1}$, auxiliary sequences $f_1, \dots, f_k : \mathbb{N}^d \rightarrow \mathbb{Q}$ with $f = f_1$, and polynomials $p_i^{(j)} \in \mathbb{Q}[y_1, \dots, y_k]$ for all $1 \leq i \leq k$ and $1 \leq j \leq d$, s.t.

$$\sigma_j f_i = p_i^{(j)}(f_1, \dots, f_k), \quad \forall 1 \leq i \leq k, 1 \leq j \leq d. \quad (7)$$

In other words, for each f_i and coordinate $1 \leq j \leq d$ there is a distinct recursive equation specifying how future values can be computed from the current one. We call systems of the form (7) *systems of polyrec equations*. Notice that the definition of a polyrec sequence contains the *promise* that $f_1, \dots, f_k$ exist solving the polyrec equations, however it is not trivial to establish whether solutions of polyrec equations exist, and thus it is not clear whether polyrec sequences have an effective syntax. We will return to this important issue in § 5.4.4.

The univariate case $d = 1$ corresponds to the polyrec sequences from [Cadilhac:Mazowiecki:Paperman:Pilipczuk:Senizergues:ToCS:2021], a rich class containing the linear recursive sequences (such as the Fibonacci numbers), as well as fast growing sequences such as 2^{2^n} . Notice that in this case every initial condition extends (uniquely) to a solution of a system of polyrec equations (7), and thus polyrec equations are an effective syntax for univariate polyrec sequences. In order to develop intuition, we show below some examples of polyrec sequences.

Example 5. We begin with a simple example, univariate and linear. The *Fibonacci sequence* $F : \mathbb{N} \rightarrow \mathbb{Q}$ is well-known to satisfy the linear recursion $F(n+2) = F(n+1) + F(n)$. We can turn this into the polyrec format by introducing an auxiliary sequence $G : \mathbb{N} \rightarrow \mathbb{Q}$ s.t. $\sigma_1 F = F + G$ and $\sigma_1 G = F$. In this way, one shows that all linear recursive sequences are polyrec.

Example 6. We now consider a bivariate example $f : \mathbb{N}^2 \rightarrow \mathbb{Q}$. For every $n_1, n_2 \in \mathbb{N}$, let $f(n_1, n_2) := 2^{2^{n_1}} \cdot 2^{3^{n_2}}$. Introducing auxiliary sequences $g(n_1, n_2) := 2^{2^{n_1}}$ and $h(n_1, n_2) := 2^{3^{n_2}}$ we have polyrec equations

$$\begin{aligned} \sigma_1 f &= f \cdot g, & \sigma_1 g &= g^2, & \sigma_1 h &= h, \\ \sigma_2 f &= f \cdot h^2, & \sigma_2 g &= g, & \sigma_2 h &= h^3. \end{aligned}$$

Example 7. The bivariate factorial $f(n_1, n_2) := n_1! \cdot n_2!$ satisfies the recursion

$$\begin{aligned} f(n_1 + 1, n_2) &= (\underline{n_1} + 1) \cdot f(n_1, n_2), \\ f(n_1, n_2 + 1) &= (\underline{n_2} + 1) \cdot f(n_1, n_2). \end{aligned}$$

This is not in the polyrec format (7), because of the two underlined occurrences of the index variables n_1, n_2 . Linear recursions with coefficients polynomial in $n_1, \dots, n_d$, as above, are called *P-recursive* [Zeilberger:JMAA:1982]. We

can introduce auxiliary sequences $g(n_1, n_2) := n_1$ and $h(n_1, n_2) := n_2$ and transform P-recursions into polyrec equations

$$\begin{aligned}\sigma_1 f &= (g + \mathbb{1}) \cdot f, & \sigma_1 g &= \mathbb{1} + g, & \sigma_1 h &= h, \\ \sigma_2 f &= (h + \mathbb{1}) \cdot f, & \sigma_2 g &= g, & \sigma_2 h &= \mathbb{1} + h.\end{aligned}$$

A similar argument shows that $n_1! + n_2!$ is polyrec. In fact, polyrec include all P -recursive definitions with a constant leading term (cf. [Lipshitz:D-finite:JA:1989]), also called *monic P-recursive* [Buna-MargineanChevalShirmohammadiWorrell:POPL:2024]. For instance, $n!$ is monic P -recursive since $(n+1)! = (n+1) \cdot n!$, but the natural recursion for the *Catalan numbers* C_n , which is $(n+2) \cdot C_{n+1} = (4n+2) \cdot C_n$, is not (the underlined term is non-constant). In fact, the Catalan numbers are known not to be polyrec [Cadilhac:Mazowiecki:Paperman:Pilipczuk:Senizergues:ToCS:2021], and thus they are not monic P -recursive either.

Remark 8. All the examples so far are sums of products of univariate polyrec sequences. For examples not of this form, consider the polyrec sequences $(n_1 + n_2)!$ and $2^{2^{n_1+n_2}}$. This shows that the characterisation of multivariate *linear* recursive sequences in [Karhumaki:TCS:1977] as sums of products of univariate linear recursive sequences does not generalise to polyrec.

More examples will be shown in § 5.4.4.

5.4.3 Closure properties of polyrec sequences. Multivariate polyrec sequences are a well-behaved class of sequences. They are closed under the algebra operations (scalar product, sum, and Hadamard product), shifts $\sigma_j f$, sections, and diagonals. The last two operations have not been defined yet. A *section* of a d -variate sequence $f : \mathbb{N}^d \rightarrow \mathbb{Q}$ is an e -variate sequence with $e < d$ which is obtained from f by fixing one or more coordinates to a constant value [Lipshitz:D-finite:JA:1989]. For instance, if $f(n_1, n_2)$ is a bivariate sequence, then $g(n_1) := f(n_1, 7)$ is a section thereof, obtained by fixing the second coordinate to be $n_2 := 7$. Since sections are defined element-wise, they commute with the operations of scalar product, sum, and Hadamard product, and with shifts not involving the coordinates being fixed. It follows that sections of polyrec sequences are polyrec.

Example 8. For instance, if $\sigma_1 f = \mathbb{1} - f^2$, then its section g satisfies the initial condition $g(0) = f(0, 7)$ and the same recursive equation $\sigma_1 g = \mathbb{1} - g^2$, preserving the polyrec format (the information about $\sigma_2 f$ is discarded when fixing the second coordinate).

A *diagonal* of a d -variate sequence $f : \mathbb{N}^d \rightarrow \mathbb{Q}$ is an e -variate sequence with $e < d$ which is obtained from f by requiring a subset of the coordinates to be equal and then by projecting them to a single coordinate [Lipshitz:D-finite:JA:1989]. For instance, the sequence $h : \mathbb{N}^2 \rightarrow \mathbb{Q}$ s.t. $h(n_1, n_3) := f(n_1, n_1, n_3)$, for all $n_1, n_3 \in \mathbb{N}$, is the diagonal of $f : \mathbb{N}^3 \rightarrow \mathbb{Q}$ obtained by identifying the first two coordinates. Again, this is an element-wise operation, and thus it commutes with the operations of scalar product, sum, Hadamard product, and of shifts not involving the coordinates being identified. Consequently, diagonals of polyrec sequences are polyrec.

Example 9. For instance, if $\sigma_1 f = \mathbb{1} - f^2$, $\sigma_2 f = f^3$, and $\sigma_3 f = \mathbb{1} + 2 \cdot f$ then the diagonal h satisfies $\sigma_1 h = (\mathbb{1} - h^2)^3$ and $\sigma_2 h = \mathbb{1} + 2 \cdot h$, preserving the polyrec format. (By composing in the other order, we get $\sigma_1 h = 1 - (h^3)^2$, but one equation suffices: If f exists, then it will satisfy both equations.)

5.4.4 Consistency of polyrec equations. So far we have defined a class of multivariate sequences, and we have argued that this class is robust by presenting examples and closure properties. However, we have sidestepped a crucial issue, namely whether the equations (7) together with an initial condition do actually define a sequence. By the shape of

the equations, a given initial condition gives rise to *at most one* solution. However, whether a solution exists at all is nontrivial.

Problem 1 (Polyrec consistency problem). *The consistency problem for polyrec equations takes as input a set of polyrec equations (7) together with an initial condition $c \in \mathbb{Q}^k$, and it amounts to decide whether there is a sequence solution $f \in (\mathbb{N}^d \rightarrow \mathbb{Q})^k$ extending the initial condition $f(0) = c$.*

We emphasise that Problem 1 is about polyrec equations, which may or may not represent a sequence, and the question is finding out which one is the case. We show that Problem 1 is decidable, thus showing that the syntax of polyrec sequences (7) is *effective*. In the univariate case $d = 1$, every initial condition extends to a solution, and thus the consistency problem is trivial. However, in the multivariate case $d \geq 2$, whether a system of equations (7) actually defines a (tuple of) sequence(s) involves an infinite number of constraints to be satisfied (cf. Figure 1). In order to develop intuition, we provide some examples illustrating the possible situations that may arise.

Example 10 (All initial conditions extend to a solution). Let $d = 2$ and consider polyrec equations

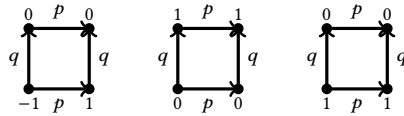
$$\sigma_1 f = f^3 \quad \text{and} \quad \sigma_2 f = f^5.$$

The update functions $p(x) := x^3$ and $q(x) := x^5$ commute identically $p(q(x)) = q(p(x)) = x^{15}$. This is a very strong condition, implying that *every* initial condition $f(0, 0) := c \in \mathbb{Q}$ extends to a sequence solution, namely $f(n_1, n_2) = c^{3^{n_1} \cdot 5^{n_2}}$.

Example 11 (Only some initial conditions extend to a solution). Revisiting Example 2, let $d = 2$ and consider polyrec equations

$$\sigma_1 f = f^2 \quad \text{and} \quad \sigma_2 f = \mathbb{1} - f^2.$$

We claim that the set of initial values extending to a solution is $\{-1, 0, 1\}$. Indeed, if we let $p(x) := x^2$ and $q(x) := 1 - x^2$, then we obtain the equation $p(q(x)) = q(p(x))$. In other words, if the initial condition x extends to a sequence solution, then it is necessarily a root of $p(q(x)) - q(p(x)) = (1 - x^2)^2 - (1 - x^4) = 2x^2(1 - x)(1 + x)$. Over $\mathbb{Q}$, the latter polynomial has precisely three solutions $x \in \{-1, 0, 1\}$. Moreover, in fact $x \in \{-1, 0, 1\}$ is also sufficient for the existence of a sequence solution, since this set is stable under $p(x)$ and $q(x)$:



Consequently, all and only $f(0, 0) \in \{-1, 0, 1\}$ extend to a solution.

Example 12 (No initial condition extends to a solution). Let $d = 2$ and consider the polyrec equations

$$\sigma_1 f = f^2 \quad \text{and} \quad \sigma_2 f = f + \mathbb{1}. \tag{8}$$

Let the initial condition be $f(0, 0) := c \in \mathbb{Q}$. From the first equation we have $f(1, 0) = c^2$, and from the second $f(0, 1) = c + 1$. But then we can obtain $f(1, 1)$ in two ways, $c^2 + 1$ and $(c + 1)^2 = c^2 + 2c + 1$, implying $c = 0$ and thus $f(1, 1) = 1$. But now we compute $f(2, 2)$ in two ways obtaining $1^2 + 1 = 2$ and $(1 + 1)^2 = 4$. This leads to a contradiction, and thus there is no initial condition extending to a solution of (8).

Problem 1 is an instance of the more general problem solvability in sequences for systems of polynomial difference equations, not necessarily of the polyrec kind (7). The latter problem is undecidable in full generality, as we now recall.

Remark 9. The more general problem of solvability in sequences for systems of polynomial difference equations is undecidable [PogudinScanlonWibmer:2020]. Nonlinear bivariate systems of two unknowns $k = d = 2$ suffice. The undecidable systems [PogudinScanlonWibmer:2020] in our notation can be written as follows:

$$\begin{aligned} (f-1)(f-2)\cdots(f-N) &= 0, \\ (g-1)(g-2)\cdots(g-N) &= 0, \end{aligned}$$

$$\prod_{i=1}^{\ell} ((a_i - f)^2 + (b_i - \sigma_2 f)^2 + (c_i - g)^2 + (d_i - \sigma_1 g)^2) = 0,$$

for suitable constants $N, a_1, \dots, a_\ell, b_1, \dots, b_\ell, c_1, \dots, c_\ell \in \mathbb{N}$, where for simplicity of notation, we have identified $n \in \mathbb{N}$ with $n \cdot \mathbb{1}$. The equations above encode plane tiling problems: the first two constraints force $f, g : \mathbb{N}^2 \rightarrow \{1, \dots, N\}$ to range over a finite set, and the last equation encodes horizontal and vertical tiling constraints. None of the equations above is in the polyrec format (7).

Consequently, in order to solve Problem 1 we need to exploit the special shape of polyrec equations. To this end, we develop a connection between sequence solutions of polyrec equations (7) and Hadamard-finite series. We make the very simple observation that number sequences and commutative series contain the same information. For instance, the sequence $2^{n_1} 3^{n_2}$ is “the same” as the series that maps $w \in \{a_1, a_2\}^*$ to $2^{\#(w)_{a_1}} 3^{\#(w)_{a_2}}$. This is made precise by saying that the difference sequence algebra $(\mathbb{N}^d \rightarrow \mathbb{Q}; \mathbb{0}, \mathbb{1}, +, \cdot, (\sigma_j)_{1 \leq j \leq d})$ and the difference Hadamard algebra of commutative series $(\mathbb{Q}\langle\langle \Sigma \rangle\rangle^{\text{comm}}; \mathbb{0}, \mathbb{1}, +, \odot, (\delta_{a_j}^L)_{1 \leq j \leq d})$ (with $\Sigma = \{a_1, \dots, a_d\}$) are isomorphic. The isomorphism maps a commutative series $f \in \mathbb{Q}\langle\langle \Sigma \rangle\rangle^{\text{comm}}$ to the number sequence $\tilde{f} : \mathbb{N}^d \rightarrow \mathbb{Q}$ s.t. $\tilde{f}(n_1, \dots, n_d) := f(a_1^{n_1} \cdots a_d^{n_d})$ for every $n_1, \dots, n_d \in \mathbb{N}$. This map preserves the difference algebra operations, which has pleasant consequences.

Given a system of polyrec equations (7), we construct a system of Hadamard-finite equations

$$\delta_{a_j}^L g_i = p_i^{(j)}(g_1, \dots, g_k), \quad \forall 1 \leq i \leq k, 1 \leq j \leq d, \quad (9)$$

which we call the *companion system* to (7).

Example 13. The polyrec equations from Example 11 $\{\sigma_1 f = f^2, \sigma_2 f = \mathbb{1} - f^2\}$ yield the companion system from Example 4 $\{\delta_{a_1}^L g = g^2, \delta_{a_2}^L g = \mathbb{1} - g^2\}$.

Every initial condition extends to a *unique* series solution $g = (g_1, \dots, g_k) \in \mathbb{Q}\langle\langle \Sigma \rangle\rangle^k$ of the companion system (9), which can be proved by induction on the length of words. By definition, this solution is a tuple of Hadamard-finite series. When all components $g_1, \dots, g_k$ are commutative, we can extract corresponding number sequences $f_1 := \tilde{g}_1, \dots, f_k := \tilde{g}_k : \mathbb{N}^d \rightarrow \mathbb{Q}$, which are precisely the unique sequence solution to the original polyrec equations (7):

$$\sigma_j f_i = \sigma_j \tilde{g}_i = \widetilde{\delta_{a_j}^L g_i} = \widetilde{p_i^{(j)}(g_1, \dots, g_k)} = p_i^{(j)}(\tilde{g}_1, \dots, \tilde{g}_k) = p_i^{(j)}(f_1, \dots, f_k). \quad \forall 1 \leq i \leq k, 1 \leq j \leq d.$$

On the other hand, when any component g_i is noncommutative, (7) does not have any number sequence solution. To see this, it suffices to notice that any sequence solution of (7) gives rise to a commutative Hadamard-finite series solution of the companion system (9) (via the inverse of the isomorphism above), but as we have observed (9) has a unique, noncommutative solution.

The argument above gives us an algorithm for the consistency problem: Construct the companion system (9) and check that all components of the unique Hadamard-finite series solution are commutative, which is decidable by Theorem 17.

Theorem 18. *The polyrec consistency problem (Problem 1) is decidable with Ackermann complexity.*

6 Shuffle automata and shuffle-finite series

In this section we study a model of computation, called shuffle automata, and the class of series they recognise. The theoretical development can be carried over in close analogy to the Hadamard case from § 5. Most of the material comes from our previous work [Clemente:CONCUR:2024] (to which we refer for more details), which here we extend by showing that the commutativity problem is decidable and we provide a nontrivial application of it. We begin in § 6.1 with some mathematical preliminaries. In § 6.2 we recall the definition of shuffle automata (previously called weighted basic parallel processes in [Clemente:CONCUR:2024]) and of shuffle-finite series. The novel technical result in this section is that the series recognised by shuffle automata are effectively closed under right derivatives (Lemma 22), and thus constitute an effective prevariety (Theorem 23). The closure property is nontrivial, since the definition of shuffle automata is asymmetric w.r.t. right and left derivatives. Together with the 2-EXPSPACE algorithm for the equivalence problem [Clemente:CONCUR:2024], we obtain that commutativity is decidable, with the same complexity (Theorem 24). We then present in § 6.3 an application to a class of multivariate power series, called *constructible differentially algebraic* (CDA), introduced by Bergeron and Sattler [BergeronSattler:TCS:1995]. We show that their syntax is effective (Theorem 26), which was an open question in our previous work [Clemente:CONCUR:2024].

6.1 Preliminaries

6.1.1 Differential algebra. A *derivation* of an algebra $(A; 0, 1, +, *)$ is a linear function $\delta : A \rightarrow A$ satisfying the following *Leibniz rule* familiar from calculus:

$$\delta(\alpha * \beta) = \delta\alpha * \beta + \alpha * \delta\beta, \quad \text{for all } \alpha, \beta \in A. \quad (\text{derivation rule})$$

In particular, this entails $\delta 1 = 0$ (by taking $\alpha = \beta = 1$). A *differential algebra* $(A; 0, 1, +, *, (\delta_j)_{1 \leq j \leq d})$ is an algebra together with finitely many derivations $\delta_1, \dots, \delta_d : A \rightarrow A$ [Ritt:DA:1950, Kaplansky:DA:1957, Kolchin:1973]. Unlike what is usually done in the literature on differential algebra, we do not require the δ_i 's to pairwise commute.

Example 14 (Differential polynomial algebra). Consider the algebra of polynomials $(\mathbb{Q}[X]; 0, 1, +, \cdot)$. A derivation $\Delta : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ acts on a polynomial by linearity and (derivation rule). For instance, $\Delta(X_1 \cdot X_2) = \Delta X_1 \cdot X_2 + X_1 \cdot \Delta X_2$ and $\Delta(X_1^5) = 5 \cdot X_1^4 \cdot \Delta X_1$. It is a basic result in differential algebra that every function $\Delta : X \rightarrow \mathbb{Q}[X]$ extends to a unique derivation $\Delta : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ [Kaplansky:DA:1957] (cf. also [BorealeCollodiGorla:ACMTCL:2024]). For instance, the familiar partial derivative $\frac{\partial}{\partial X_i}$ (for short, ∂_{X_i}) is the unique derivation Δ s.t. $\Delta X_i = 1$ and $\Delta X_j = 0$ for $j \neq i$. If we fix distinguished derivations $\Delta_1, \dots, \Delta_d$, then we obtain the polynomial differential algebra $(\mathbb{Q}[X]; 0, 1, +, \cdot, (\Delta_j)_{1 \leq j \leq d})$. Polynomial differential algebras will provide the state space of configurations for shuffle automata. Other examples of differential algebras will be presented in § 6.1.2 and § 6.3.

6.1.2 *Shuffle differential algebra.* We recall a product operation on series which will be central in the development of this section. The *shuffle product* of two series $f \sqcup g$ is defined coinductively by (cf. [BasoldHansenPinRutten:MSCS:2017])

$$[\varepsilon](f \sqcup g) = f_\varepsilon \cdot g_\varepsilon, \quad (\sqcup-\varepsilon)$$

$$\delta_a^L(f \sqcup g) = \delta_a^L f \sqcup g + f \sqcup \delta_a^L g, \quad \forall a \in \Sigma. \quad (\sqcup-\delta_a^L)$$

This is an associative, commutative, and bilinear operation, with identity the series $1 \cdot \varepsilon$ mapping ε to 1 and zero everywhere else. Shuffle product originates in the work of Eilenberg and MacLane in homological algebra [EilenbergMacLane:AM:1953], and was introduced in automata theory by Fliess under the name of *Hurwitz product* [Fliess:1974]. It is the series analogue of the shuffle product in language theory. It finds applications in concurrency theory, where it models the *interleaving semantics* of process composition, and in control theory, where it models the combinatorics of iterated integrals [Fliess:1981]. Intuitively, it models all possible ways in which two sequences can be interleaved.

By $(\sqcup-\delta_a^L)$, left derivatives δ_a^L ($a \in \Sigma$) are derivations for the shuffle product, and we have thus obtained the *differential shuffle algebra* $\mathbb{Q}\langle\langle\Sigma\rangle\rangle_\sqcup := (\mathbb{Q}\langle\langle\Sigma\rangle\rangle; +, \sqcup, (\delta_a^L)_{a \in \Sigma})$. It is worth noting that right derivatives are also derivations of the shuffle algebra. This will be useful in the proof of Lemma 22.

Lemma 19. *Right derivatives δ_a^R (with $a \in \Sigma$) are shuffle algebra derivations. I.e., they are linear and satisfy Leibniz rule (derivation rule) for the shuffle product:*

$$\delta_a^R(f \sqcup g) = \delta_a^R f \sqcup g + f \sqcup \delta_a^R g, \quad \forall a \in \Sigma. \quad (\sqcup-\delta_a^R)$$

PROOF. We have already observed linearity in § 2. Regarding $(\sqcup-\delta_a^R)$, we prove it by coinduction. First of all, the constant term of both sides is the same, since

$$[\varepsilon]\delta_a^R(f \sqcup g) = [a](f \sqcup g) = f_a \cdot g_\varepsilon + f_\varepsilon \cdot g_a, \text{ and}$$

$$[\varepsilon](\delta_a^R f \sqcup g + f \sqcup \delta_a^R g) = [\varepsilon](\delta_a^R f \sqcup g) + [\varepsilon](f \sqcup \delta_a^R g) = [\varepsilon]\delta_a^R f \cdot [\varepsilon]g + [\varepsilon]f \cdot [\varepsilon]\delta_a^R g = f_a \cdot g_\varepsilon + f_\varepsilon \cdot g_a.$$

The proof is concluded by showing that, for every $b \in \Sigma$, the b -left derivative δ_b^L of both sides is the same:

$$\delta_b^L \delta_a^R(f \sqcup g) = \delta_b^L(\delta_a^R f \sqcup g + f \sqcup \delta_a^R g).$$

We indeed have

$$\begin{aligned} \delta_b^L \delta_a^R(f \sqcup g) &= \\ &= \delta_a^R \delta_b^L(f \sqcup g) && \text{(by Lemma 3)} \\ &= \delta_a^R(\delta_b^L f \sqcup g + f \sqcup \delta_b^L g) && \text{(by } (\sqcup-\delta_b^L)) \\ &= \delta_a^R(\delta_b^L f \sqcup g) + \delta_a^R(f \sqcup \delta_b^L g) && \text{(by linearity)} \\ &= \delta_a^R \delta_b^L f \sqcup g + \delta_b^L f \sqcup \delta_a^R g + \delta_a^R f \sqcup \delta_b^L g + f \sqcup \delta_a^R \delta_b^L g && \text{(by coinduction)} \\ &= \delta_b^L \delta_a^R f \sqcup g + \delta_b^L f \sqcup \delta_a^R g + \delta_a^R f \sqcup \delta_b^L g + f \sqcup \delta_b^L \delta_a^R g && \text{(by Lemma 3)} \\ &= \delta_b^L(\delta_a^R f \sqcup g) + \delta_b^L(f \sqcup \delta_a^R g) && \text{(by } (\sqcup-\delta_b^L)) \\ &= \delta_b^L(\delta_a^R f \sqcup g + f \sqcup \delta_a^R g) && \text{(by linearity).} \quad \square \end{aligned}$$

6.2 Shuffle automata, shuffle-finite series, and their commutativity problem

In this section we recall the definition of shuffle automata, the series they recognise, and study their semantical and computational properties. Shuffle automata have been introduced in [Clemente:CONCUR:2024] under the name of *weighted basic parallel processes*, a simultaneous generalisation of weighted finite automata and a model of concurrency called *basic parallel processes* [Esparza:FI:1997]. For reasons of uniformity with the rest of the paper, we shall refer to them as shuffle automata. Shuffle automata find applications also in control theory [Clemente:control:arXiv:2025], however we will not explore this topic here.

6.2.1 Shuffle automata. Syntactically, a *shuffle automaton* $\mathcal{A} = (\Sigma, X, F, \Delta)$ is identical to a Hadamard automaton (cf. § 5.3), however the semantics is different: The transition function $\Delta_a : X \rightarrow \mathbb{Q}[X]$ (for $a \in \Sigma$) is extended to a unique derivation $\Delta_a : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ of the differential polynomial algebra of configurations (cf. (derivation rule)). We have seen in Example 14 that such an extension exists and is unique. The extension to all finite words and the definition of the semantics of a configuration are unchanged (cf. (2)). The series recognised by the shuffle automaton $\mathcal{A}$ is $\mathcal{A}[\![X]\!]$.

Example 15. Consider a binary alphabet $\Sigma = \{a_1, a_2\}$ and three nonterminals $X = \{X_1, X_2, X_3\}$. Let the output function be $FX_1 := 1$ and $FX_2 := FX_3 := 0$, and consider transitions

$$\begin{aligned} \Delta_{a_1}X_1 &:= X_1 \cdot (1 + X_3), & \Delta_{a_1}X_2 &:= 1, & \Delta_{a_1}X_3 &:= 0, \\ \Delta_{a_2}X_1 &:= X_1 \cdot X_2, & \Delta_{a_2}X_2 &:= 0, & \Delta_{a_2}X_3 &:= 1. \end{aligned}$$

The run starting from X_1 and reading input $w = a_1a_2$ is

$$X_1 \xrightarrow{a_1} X_1 \cdot (1 + X_3) \xrightarrow{a_2} \Delta_{a_2}(X_1 \cdot (1 + X_3)) = \Delta_{a_2}X_1 \cdot (1 + X_3) + X_1 \cdot \Delta_{a_2}(1 + X_3) = X_1 \cdot X_2 \cdot (1 + X_3) + X_1.$$

and thus $\mathcal{A}[\![X]\!]_{a_1a_2} = F(X_1 \cdot X_2 \cdot (1 + X_3) + X_1) = 1$. We will see in Example 16 that the series recognised by X_1 maps w to $\binom{n}{k} \cdot k!$ where $n := \#(w)_{a_1}$ and $k := \#(w)_{a_2}$.

We now recall some basic properties of shuffle automata. A shuffle automaton endows the algebra of configurations with the structure of a differential algebra $(\mathbb{Q}[X]; 0, 1, +, \cdot, (\Delta_a)_{a \in \Sigma})$. The following connection with the differential algebra of series bears similarity to Lemma 14.

Lemma 20 (Properties of the semantics [Clemente:CONCUR:2024]). *The semantics of a shuffle automaton is a homomorphism from the differential algebra of polynomials $(\mathbb{Q}[X]; +, \cdot, (\Delta_a)_{a \in \Sigma})$ to the differential shuffle algebra of series $(\mathbb{Q}\langle\langle\Sigma\rangle\rangle; +, \sqcup, (\delta_a^L)_{a \in \Sigma})$. In other words, $\llbracket 0 \rrbracket = 0$, $\llbracket 1 \rrbracket = 1 \cdot \varepsilon$, and, for all configurations $\alpha, \beta \in \mathbb{Q}[X]$,*

$$\llbracket c \cdot \alpha \rrbracket = c \cdot \llbracket \alpha \rrbracket \quad \forall c \in \mathbb{Q}, \quad (10)$$

$$\llbracket \alpha + \beta \rrbracket = \llbracket \alpha \rrbracket + \llbracket \beta \rrbracket, \quad (11)$$

$$\llbracket \alpha \cdot \beta \rrbracket = \llbracket \alpha \rrbracket \sqcup \llbracket \beta \rrbracket, \quad (12)$$

$$\llbracket \Delta_a \alpha \rrbracket = \delta_a^L \llbracket \alpha \rrbracket \quad \forall a \in \Sigma. \quad (13)$$

6.2.2 Shuffle-finite series. Shuffle automata are data structures representing series. The same class of series admits a semantic presentation, which we find more convenient to work with and which we now recall. For series $f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, we denote by $\mathbb{Q}[f_1, \dots, f_k]_{\sqcup}$ the smallest shuffle algebra containing $f_1, \dots, f_k$. Algebras of this form are called *finitely generated*, with *generators* $f_1, \dots, f_k$. When it is closed under the derivations δ_a^L (for all $a \in \Sigma$), it is called a *differential shuffle algebra*. A series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is *shuffle finite* [Clemente:CONCUR:2024] if it belongs to a finitely generated

differential shuffle algebra (note the analogy with Hadamard-finite series, cf. Definition 1). In practice, we will be using the following equivalent definition of shuffle finiteness.

Lemma 21 ([Clemente:WBPP:arXiv:2024]). *A series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is shuffle finite iff there are generators $f = f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ s.t.:*

- (1) $f \in \mathbb{Q}[f_1, \dots, f_k]_{\sqcup}$, and
- (2) $\delta_a^L f_i \in \mathbb{Q}[f_1, \dots, f_k]_{\sqcup}$, for all $a \in \Sigma$ and $1 \leq i \leq k$.

Thanks to this characterisation, we prove that shuffle-finite series coincide with the series recognised by shuffle automata [Clemente:CONCUR:2024], and constitute an effective differential algebra [Clemente:CONCUR:2024]. We complete the picture by observing that they are effectively closed under right derivatives.

Lemma 22. *If f is shuffle finite, then $\delta_a^R f$ is also effectively shuffle finite.*

The definition of shuffle-finite series is asymmetric since it is biased towards left derivatives, therefore closure under right derivatives is nontrivial. Nonetheless, the technology of shuffle-finite series, together with the fact that left and right derivatives commute (Lemma 3), gives a rather short proof of this fact.

PROOF. The proof is similar to the one of Lemma 13. Let f be shuffle finite and let $f_1, \dots, f_k$ be the generators provided by Lemma 21. By the first property, $f \in A := \mathbb{Q}[f_1, \dots, f_k]_{\sqcup}$. Consider the shuffle algebra B generated by the f_i 's and their right derivatives, $B := \mathbb{Q}[f_1, \dots, f_k, \delta_a^R f_1, \dots, \delta_a^R f_k]_{\sqcup}$.

Claim. *For every series $g \in A$, we have $\delta_a^R g \in B$.*

PROOF. We proceed by induction on polynomial expressions. In the base case, we have $g = f_i \in A$ and there is nothing to prove. In the case $g = g_1 + g_2$ with $g_1, g_2 \in A$, we have $\delta_a^R g = \delta_a^R g_1 + \delta_a^R g_2$ and the claim follows from the inductive assumption applied to $\delta_a^R g_1, \delta_a^R g_2$. Finally, consider $g = g_1 \sqcup g_2$ with $g_1, g_2 \in A$. By the product rule for right derivatives ($\sqcup$ - δ_a^R), we have $\delta_a^R g = \delta_a^R g_1 \sqcup g_2 + g_1 \sqcup \delta_a^R g_2$. By the inductive assumption, $\delta_a^R g_1, \delta_a^R g_2 \in B$, and thus the same holds for $\delta_a^R g$. $\square$

Since by assumption $f \in A$, by the claim above we have $\delta_a^R f \in B$. We need to show that B is differential, which, by the second point of Lemma 21, amounts to prove that left derivatives of generators are themselves in B . Left derivatives of the old generators f_i are in A (since A is differential by assumption), and thus they are in the larger B . It remains to show the same for the new generators $\delta_a^R f_i$. So consider $\delta_b^L \delta_a^R f_i$ for an arbitrary $b \in \Sigma$ and we have to show that it is in B . By Lemma 3 left and right derivations commute, and thus we have $\delta_a^L \delta_b^R f_i = \delta_b^R \delta_a^L f_i$. But by assumption $\delta_a^L f_i \in A$, therefore by the claim $\delta_b^R \delta_a^L f_i \in B$, as required. $\square$

We obtain the following synthesis of the above-mentioned results from [Clemente:CONCUR:2024] and Lemma 22.

Theorem 23. *The class of shuffle-finite series is an effective prevariety.*

From Theorems 1 and 23, it follows that the commutativity problem reduces efficiently to the equality problem. Moreover, it is easy to see that shuffle-finite series are closed under anti-derivatives (same proof as in Lemma 6), and the new generators are computable in polynomial time. Consequently, commutativity and equality are efficiently inter-reducible. Since the equality problem for shuffle-finite series is in 2-EXSPACE [Clemente:CONCUR:2024], the same complexity upper bound applies to the commutativity problem, which is the main result of the section.

Theorem 24. *The commutativity problem for shuffle-finite series (equivalently, series recognised by shuffle automata) is decidable in 2-EXSPACE.*

As directly inherited from the equality problem, the complexity upper bound is doubly exponential in the number of generators k .

6.3 Application: Multivariate constructible differentially algebraic power series

In this section, we recall the definition of a rich class of multivariate series in commuting indeterminates called *constructible differentially algebraic* (CDA). Based on Theorem 24, we show that they have an effective syntax, addressing a problem left open in [Clemente:CONCUR:2024]. We begin in § 6.3.1 with some preliminaries, then in § 6.3.2 we recall the definition of CDA, and finally we show effectiveness in § 6.3.3.

6.3.1 Differential power series algebra. In the rest of the section, fix a dimension $d \in \mathbb{N}_{\geq 1}$ and consider d pairwise commuting independent variables $x = (x_1, \dots, x_d)$. For a multi-index $n = (n_1, \dots, n_d) \in \mathbb{N}^d$, we write x^n for $x_1^{n_1} \cdots x_d^{n_d}$, and $n!$ for $n_1! \cdots n_d!$. An *exponential multivariate power series* (in commuting variables) is a function $f : \mathbb{N}^d \rightarrow \mathbb{Q}$, customarily written as $f = \sum_{n \in \mathbb{N}^d} f_n \cdot \frac{x^n}{n!}$ where $f_n \in \mathbb{Q}$ is the coefficient of term x^n . We write the set of power series as $\mathbb{Q}[[x]]$. For example $1 + x_1 + x_1^2 + \cdots = \frac{1}{1-x_1}$ is a power series. We endow the set of power series with the structure of a vector space with zero 0, scalar product $c \cdot f$, and addition $f + g$ all defined element-wise. Therefore, the vector spaces of power series and number sequences (cf. § 5.4.1) are isomorphic. However, multiplication is different. While multiplication of number sequences is element-wise (Hadamard product), for power series we define multiplication in the same way as for multiplication of polynomials. In other words, if $f = \sum_{n \in \mathbb{N}^d} f_n \cdot \frac{x^n}{n!}$ and $g = \sum_{k \in \mathbb{N}^d} g_k \cdot \frac{x^k}{k!}$, then

$$f \cdot g = \sum_{n, k \in \mathbb{N}^d} f_n g_k \cdot \frac{x^{n+k}}{n!k!} = \sum_{m \in \mathbb{N}^d} \left(\sum_{n+k=m} \binom{m}{n, k} f_n g_k \right) \cdot \frac{x^m}{m!},$$

where $\binom{m}{n, k}$ denotes the multivariate binomial coefficient $\frac{m!}{n!k!}$. At the level of sequences of coefficients, the formula above illustrates the the product of polynomials corresponds to the binomial convolution product. Finally, for every $1 \leq j \leq d$ the j -th partial derivative ∂_{x_j} maps the power series f to the power series $\partial_{x_j} f = \sum_{n \in \mathbb{N}^d} f_{n+e_j} \frac{x^n}{n!}$. Notice that ∂_{x_j} is a derivation with respect to the product of power series, since it satisfies the familiar Leibniz rule from calculus (cf. (derivation rule)):

$$\partial_{x_j}(f \cdot g) = (\partial_{x_j} f) \cdot g + f \cdot (\partial_{x_j} g).$$

For example, $\partial_{x_1} \frac{1}{1-x_1} = \frac{1}{(1-x_1)^2}$. We have thus defined the *differential power series algebra* $(\mathbb{Q}[[x]]; 0, 1, +, \cdot, (\partial_{x_j})_{1 \leq j \leq d})$. Notice that in this case, unlike in the case of the differential shuffle series algebra in § 6.1.2, the derivatives are pairwise commuting $\partial_{x_j} \circ \partial_{x_h} = \partial_{x_h} \circ \partial_{x_j}$. The differential power series algebra will constitute the semantic background for the developments in the rest of the section.

6.3.2 CDA power series. Let y denote a tuple of commuting indeterminates $y = (y_1, \dots, y_k)$. A power series $f \in \mathbb{Q}[[x]]$ is CDA [BergeronReutenauer:EJC:1990, BergeronSattler:TCS:1995] if there exist $k \in \mathbb{N}_{\geq 1}$ and auxiliary power series $f_1, \dots, f_k \in \mathbb{Q}[[x]]$ with $f = f_1$ satisfying a system of polynomial partial differential equations

$$\partial_{x_j} f_i = p_i^{(j)}(f_1, \dots, f_k), \quad \forall 1 \leq i \leq k, 1 \leq j \leq d, \quad (14)$$

where $p_i^{(j)} \in \mathbb{Q}[y]$ for all $1 \leq i \leq k$ and $1 \leq j \leq d$. We call systems of the form above *systems of autonomous CDA equations*.

Remark 10 (Autonomous vs. non-autonomous). We could allow the r.h.s. of equations to contain the independent variables x_j 's, i.e., $p_i^{(j)} \in \mathbb{Q}[x, y]$, obtaining *non-autonomous* equations. This does not increase expressiveness since we can introduce fresh power series $g_j := x_j$ together with equations $\partial_{x_j} g_j = 1$, $\partial_{x_h} g_j = 0$ for $h \neq j$, and initial conditions $g_j(0) = 0$. For this reason, we focus on autonomous equations.

CDA power series are commonly found in combinatorics. For example, they include all algebraic power series (i.e., power series solutions of polynomial equations [KuichSalomaa:1986], e.g., $\sqrt{1-x^2}$), the exponential power series e^x , the trigonometric power series $\sin x$, $\cos x$, and many others. See [BergeronReutenauer:EJC:1990, BergeronSattler:TCS:1995] and [Clemente:CONCUR:2024] for additional examples. Additionally, CDA power series are closed under natural operations, such as scalar product, addition, multiplication, differentiation, and composition (cf. [Clemente:CONCUR:2024]). They find applications in combinatorics, since the generating functions of a large class of finite structures, called constructible species, are CDA [Clemente:CONCUR:2024]. Finally, the CDA equivalence problem is decidable in 2-EXPTIME, thus of elementary complexity [Clemente:CONCUR:2024]; this should be contrasted with polyrec sequences, for which the best upper bound is Ackermannian § 5.4.2.

Example 16 (Binomial coefficients). Consider the mixed generating power series of the binomial coefficients $f_1 := \sum_{n,k \in \mathbb{N}} \binom{n}{k} \cdot \frac{x_1^n}{n!} \cdot x_2^k = e^{x_1 \cdot (1+x_2)}$. By introducing auxiliary power series $f_2 := x_1$ and $f_3 := x_2$, we obtain the CDA equations:

$$\begin{aligned} \partial_{x_1} f_1 &= f_1 \cdot (1 + f_3), & \partial_{x_1} f_2 &= 1, & \partial_{x_1} f_3 &= 0, \\ \partial_{x_2} f_1 &= f_1 \cdot f_2, & \partial_{x_2} f_2 &= 0, & \partial_{x_2} f_3 &= 1. \end{aligned}$$

Example 17 (Stirling numbers of the second kind [BergeronSattler:TCS:1995]). Consider the mixed generating power series of the Stirling numbers of the second kind $f_1 = \sum_{n,k \in \mathbb{N}} S_{n,k} \cdot \frac{x_1^n}{n!} \cdot x_2^k = e^{x_1 \cdot (e^{x_2} - 1)}$. By introducing auxiliary power series $f_2 = e^{x_2}$ and $f_3 = x_1$ we obtain the following CDA equations

$$\begin{aligned} \partial_{x_1} f_1 &= f_1 \cdot (f_2 - 1), & \partial_{x_1} f_2 &= 0, & \partial_{x_1} f_3 &= 1, \\ \partial_{x_2} f_1 &= f_1 \cdot f_2 \cdot f_3, & \partial_{x_2} f_2 &= f_2, & \partial_{x_2} f_3 &= 0. \end{aligned}$$

We will come back to these examples in the following section.

6.3.3 Solvability in power series of CDA equations. The definition of CDA power series relies on the *promise* that the system differential equations (14) does have a power series solution starting at a given initial value. Deciding whether this is the case is a nontrivial problem, left open in [Clemente:CONCUR:2024].

Problem 2 (CDA solvability problem). *The solvability problem for CDA equations takes as input a set of CDA equations (14) together with an initial condition $c \in \mathbb{Q}^k$, and it amounts to decide whether there is a power series solution $f \in \mathbb{Q}[[x]]^k$ extending the initial condition $f(0) = c$.*

We remark that the initial condition is part of the input to the solvability problem. In the univariate case $d = 1$, by the classic Picard-Lindelöf theorem [CoddingtonLevinson:1984], every initial condition extends to a solution, and thus the solvability problem is trivial. The multivariate case $d \geq 2$ is nontrivial, as the following examples demonstrate.

Example 18 (Unsolvable systems). A CDA system may not have any power series solution at all when $d \geq 2$, regardless of the initial condition. For instance, the following equations are unsatisfiable in power series:

$$\begin{aligned} \partial_{x_1} f = 0, & \quad \text{and} \quad \partial_{x_1} g = 1, \\ \partial_{x_2} f = g, & \quad \partial_{x_2} g = 1. \end{aligned}$$

The equation $\partial_{x_1} g = 1$ implies $g = x_1 + c(x_2)$ for some $c \in \mathbb{Q}[[x_2]]$, and $\partial_{x_2} g = 1$ implies $g = x_2 + d(x_1)$ for some $d \in \mathbb{Q}[[x_1]]$. Taken together, we have $g = a + x_1 + x_2$ for some $a \in \mathbb{Q}$. The equation $\partial_{x_1} f = 0$ implies $f \in \mathbb{Q}[[x_2]]$, and thus $\partial_{x_2} f = g \in \mathbb{Q}[[x_2]]$, which is a contradiction.

Example 19 (Sensitivity to the initial condition). Existence of power series solutions may depend on the initial value when $d \geq 2$. Consider equations

$$\begin{aligned} \partial_{x_1} f = f + g, & \quad \text{and} \quad \partial_{x_1} g = 0, \\ \partial_{x_2} f = 0, & \quad \partial_{x_2} g = g. \end{aligned}$$

Thus $f \in \mathbb{Q}[[x_1]]$, and $g = a \cdot e^{x_2+b} \in \mathbb{Q}[[x_2]]$, for some $a, b \in \mathbb{Q}$. Since $g = \partial_{x_1} f - f \in \mathbb{Q}[[x_2]]$, there are no solutions with $g(0) = a \cdot e^b \neq 0$. When $g(0) = 0$ (iff $a = 0$), we have the family of solutions $g = 0$ and $f = c \cdot e^{x_1+d}$ for every $c, d \in \mathbb{Q}$.

We now remark that solvability in power series of systems of polynomial partial differential equations, not necessarily in the special CDA format, is undecidable. This should be compared to the analogous undecidability result about systems of polynomial difference equations (Remark 9). While undecidability for sequences was based on tiling problems, for differential equations it is based on diophantine equations.

Remark 11. A seminal result of Denef and Lipshitz shows that solvability in power series for polynomial partial differential equations is *undecidable* [DenefLipshitz:1984]. In fact, this holds already for a *single, linear* partial differential equation in *one unknown*, with polynomial coefficients in $\mathbb{Q}[x]$. In order to gain insight into the shape of the undecidable equations, we present their reduction. Let $P(y_1, \dots, y_d) \in \mathbb{Z}[y_1, \dots, y_d]$ be a multivariate polynomial with integer coefficients. Consider the non-autonomous differential equation

$$P(x_1 \cdot \partial_{x_1}, \dots, x_d \cdot \partial_{x_d})f = \frac{1}{1-x_1} \cdots \frac{1}{1-x_d}, \quad (15)$$

with initial condition $f(0) = P(0)^{-1}$ (assuming $P(0) \neq 0$). For example, if $P(y_1, y_2) = 1 - 2 \cdot y_1 + y_2^2$ then we would get the equation $(1 - 2 \cdot x_1 \cdot \partial_{x_1} + (x_2 \cdot \partial_{x_2})^2)f = \frac{1}{1-x_1} \frac{1}{1-x_2}$ where $(x_2 \cdot \partial_{x_2})^2$ is the derivation $x_2 \cdot \partial_{x_2}$ applied twice. Equation (15) has a solution in power series iff $P = 0$ has no nonnegative integer solutions: Indeed, the r.h.s. is just $\sum_{n \in \mathbb{N}^d} x^n$ and if $f \in \mathbb{C}[[x]]$ is the power series $f = \sum_{n \in \mathbb{N}^d} f_n \cdot x^n$, then the l.h.s. equals $\sum_{n \in \mathbb{N}^d} f_n \cdot P(n) \cdot x^n$. As a consequence of Hilbert 10th problem (cf. [Matijasevic:2003]), whether $P = 0$ has a nonnegative integer solution is undecidable (already for $d = 9$), and thus solvability in power series for (15) is undecidable as well.

Note that (15) is not in the CDA format. One reason is that it is non-autonomous since it contains occurrences of the independent variables x_j 's. This is inessential and it can be addressed by Remark 10. Another reason concerns the occurrence of the rational power series $\frac{1}{1-x_j}$ on the r.h.s. of (15), however this can be avoided by multiplying both sides of the equation by $(1-x_1) \cdots (1-x_d)$. The last reason is more fundamental: Both $\partial_{x_j} f$ and $\partial_{x_h} f$ (with $j \neq h$) appear in the same equation, which in CDA is not allowed. Consequently, in order to decide Problem 2 we need to exploit the CDA format. To this end we develop a connection between power series solutions of CDA equations (14)

and shuffle automata, in a way analogous to the development in § 5.4.4 of a connection between multivariate polyrec sequences and Hadamard automata.

Consider an alphabet of noncommuting indeterminates $\Sigma = \{a_1, \dots, a_d\}$. We observe that the differential algebra of power series and the differential shuffle algebra of commutative series,

$$(\mathbb{Q}[[x]]; 0, 1, +, \cdot, (\partial_{x_j})_{1 \leq j \leq d}), \quad \text{resp.}, \quad (\mathbb{Q}\langle\langle\Sigma\rangle\rangle^{\text{comm}}; 0, 1 \cdot \varepsilon, +, \sqcup, (\delta_{a_j}^L)_{1 \leq j \leq d})$$

are isomorphic. The isomorphism maps a commutative series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle^{\text{comm}}$ to the power series $\widetilde{f} = \sum_{n \in \mathbb{N}^d} \widetilde{f}_n \cdot \frac{x^n}{n!} \in \mathbb{Q}[[x]]$ with coefficients $\widetilde{f}_{n_1, \dots, n_d} := f(a_1^{n_1} \cdots a_d^{n_d})$ for every $n_1, \dots, n_d \in \mathbb{N}$. In other words, this is a bijective mapping preserving the algebraic and differential structure, i.e., $\widetilde{0} = 0$, $\widetilde{1 \cdot \varepsilon} = 1$, and

$$\begin{aligned} \widetilde{c \cdot f} &= c \cdot \widetilde{f}, & \widetilde{f \sqcup g} &= \widetilde{f} \cdot \widetilde{g}, \\ \widetilde{f + g} &= \widetilde{f} + \widetilde{g}, & \widetilde{\delta_{a_j}^L f} &= \partial_{x_j} \widetilde{f}, \end{aligned} \quad (16)$$

for all $f, g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle^{\text{comm}}$, $c \in \mathbb{Q}$, and $1 \leq j \leq d$. For a system of CDA equations (14), together with an initial condition $c = (c_1, \dots, c_k) \in \mathbb{Q}^k$, we construct its *companion shuffle automaton* (Σ, X, F, Δ) , which has nonterminals $X = (X_1, \dots, X_k)$, output function $FX_i := c_i$ for all $1 \leq i \leq k$, and transitions

$$\Delta_{a_j} X_i := p_i^{(j)}(X_1, \dots, X_k), \quad \forall 1 \leq i \leq k, 1 \leq j \leq d. \quad (17)$$

Let $f_i := [X_i]$ be the series recognised by nonterminal X_i of the automaton, for all $1 \leq i \leq k$, and write $f = (f_1, \dots, f_k)$. The following lemma provides a characterisation of power series solvability of (14) in terms of the series recognised by the shuffle automaton.

Lemma 25. *The initial condition c extends to a power series solution of the CDA equations (14) if, and only if, f is a tuple of commutative series.*

PROOF. On the one hand, if f is a tuple of commutative series with initial condition $[\varepsilon]f = c$ solving (17), then, by following the isomorphism, $\widetilde{f} \in \mathbb{Q}[[x]]^k$ is a tuple of power series solutions of (14) with initial condition $f(0) = c$. Indeed, by the properties of the semantics of shuffle automata (Lemma 20), component f_i satisfies differential equations

$$\delta_{a_j}^L f_i = p_i^{(j)}(f_1, \dots, f_k), \quad \forall 1 \leq i \leq k, 1 \leq j \leq d.$$

By applying the isomorphism to both sides of the equation above, by (16) we get

$$\partial_{x_j} \widetilde{f}_i = \widetilde{\delta_{a_j}^L f_i} = \widetilde{p_i^{(j)}(f_1, \dots, f_k)} = p_i^{(j)}(\widetilde{f}_1, \dots, \widetilde{f}_k),$$

and thus $\widetilde{f}$ satisfies the CDA equations (14). On the other hand, if $g \in \mathbb{Q}[[x]]^k$ is a tuple of power series solutions of (14) with initial condition $g(0) = c$, then by following the inverse of the isomorphism we obtain a tuple of commutative series $h \in (\mathbb{Q}\langle\langle\Sigma\rangle\rangle^{\text{comm}})^k$ s.t. $\widetilde{h} = g$ and $[\varepsilon]h = c$, solving equations (17). Since equations (17) admit exactly one series solution with c as initial condition, $h = f$ is the semantics of the shuffle automaton. Consequently, f is a tuple of commutative series. $\square$

We illustrate the characterisation provided by Lemma 25 on two simple examples.

Example 20 (Uniformly solvable systems). Consider the CDA equations from Example 16. The companion shuffle automaton is the one from Example 15. We claim that $\Delta_{a_1} \circ \Delta_{a_2} = \Delta_{a_2} \circ \Delta_{a_1}$. It suffices to verify the claim for the

generators X_1, X_2, X_3 :

$$\Delta_{a_1} \Delta_{a_2} X_1 = \Delta_{a_1} (X_1 \cdot X_2) = X_1 \cdot (1 + X_3) \cdot X_2 + X_1 = X_1 + X_1 X_2 + X_1 X_2 X_3,$$

$$\Delta_{a_2} \Delta_{a_1} X_1 = \Delta_{a_2} (X_1 \cdot (1 + X_3)) = X_1 \cdot X_2 \cdot (1 + X_3) + X_1 = X_1 + X_1 X_2 + X_1 X_2 X_3,$$

$$\Delta_{a_1} \Delta_{a_2} X_2 = \Delta_{a_2} \Delta_{a_1} X_2 = 0,$$

$$\Delta_{a_1} \Delta_{a_2} X_3 = \Delta_{a_2} \Delta_{a_1} X_3 = 0.$$

Thus, every initial condition $c \in \mathbb{Q}^3$ yields commutative series $\llbracket X \rrbracket = (\llbracket X_1 \rrbracket, \llbracket X_2 \rrbracket, \llbracket X_3 \rrbracket) \in \mathbb{Q}\langle\langle a_1, a_2 \rangle\rangle^3$ starting at $[e]\llbracket X \rrbracket = c$. Thanks to Lemma 25, this means that every initial condition $c \in \mathbb{Q}^3$ extends to a power series solution $f \in \mathbb{Q}\llbracket x_1, x_2, x_3 \rrbracket$ starting at $f(0) = c$. This shows that Example 16 is *uniformly solvable*. The solution $(e^{x_1 \cdot (1+x_2)}, x_1, x_2)$ given there corresponds to the initial condition $c = (1, 0, 0)$.

Example 21. The CDA equations from Example 17 are also uniformly solvable. To see this consider the companion shuffle automaton. It has three nonterminals $X = \{X_1, X_2, X_3\}$ and transitions

$$\begin{aligned} \Delta_{a_1} X_1 &:= X_1(X_2 - 1), & \Delta_{a_1} X_2 &:= 0, & \Delta_{a_1} X_3 &:= 1, \\ \Delta_{a_2} X_1 &:= X_1 X_2 X_3, & \Delta_{a_2} X_2 &:= X_2, & \Delta_{a_2} X_3 &:= 0. \end{aligned}$$

As in the previous example, $\Delta_{a_1} \circ \Delta_{a_2} = \Delta_{a_2} \circ \Delta_{a_1}$, since it holds for the generators $\{X_1, X_2, X_3\}$:

$$\Delta_{a_1} \Delta_{a_2} X_1 = \Delta_{a_1} (X_1 X_2 X_3) = X_1 (X_2 - 1) X_2 X_3 + X_1 X_2$$

$$\Delta_{a_2} \Delta_{a_1} X_1 = \Delta_{a_2} (X_1 (X_2 - 1)) = X_1 X_2 X_3 (X_2 - 1) + X_1 X_2$$

$$\Delta_{a_1} \Delta_{a_2} X_2 = \Delta_{a_2} \Delta_{a_1} X_2 = 0$$

$$\Delta_{a_1} \Delta_{a_2} X_3 = \Delta_{a_2} \Delta_{a_1} X_3 = 0.$$

Consequently, every initial condition $c \in \mathbb{Q}^3$ yields a commutative series $\llbracket X \rrbracket \in \mathbb{Q}\langle\langle a_1, a_2 \rangle\rangle^3$ starting at $\llbracket X \rrbracket = c$, and by Lemma 25 this means that every initial condition $c \in \mathbb{Q}^3$ extends to a power series solution of the CDA equations in Example 17.

We now have all ingredients to solve Problem 2. For CDA equations (14) and initial condition $c \in \mathbb{Q}^k$ we build their companion shuffle automaton (cf. (17)), and check that all series $\llbracket X_1 \rrbracket, \dots, \llbracket X_k \rrbracket$ recognised by its nonterminals are commutative. The latter is decidable in 2-EXSPACE by Theorem 24. Correctness follows from Lemma 25.

Theorem 26. *The CDA solvability problem (Problem 2) is decidable in 2-EXSPACE.*

7 Infiltration-finite series and infiltration automata

We have so far considered the Hadamard (§ 5) and the shuffle products (§ 6), motivated by their applications to polyrec sequences (§ 5.4.2), resp., CDA power series (§ 6.3.2). Besides Hadamard and shuffle, in the literature one finds another product, called *infiltration* (§ 7.1). We observe that the infiltration product falls under the scope of our techniques. In a manner analogous to the Hadamard and shuffle products, we can define *infiltration-finite series* (§ 7.2) and corresponding *infiltration automata* (§ 7.3), which are novel models of computation not previously investigated. The main result of this section (§ 7.4) is that infiltration-finite series constitute an effective prevariety and thus the commutativity problem is decidable for this class (Theorem 37).

7.1 Preliminaries

7.1.1 *Infiltration algebras.* An *infiltration function* of an algebra $(A; 0, 1, +, *)$ (in short, *infiltration*) is a linear function $\delta : A \rightarrow A$ satisfying the following product rule

$$\delta(\alpha * \beta) = \delta\alpha * \beta + \alpha * \delta\beta + \delta\alpha * \delta\beta, \quad \text{for all } \alpha, \beta \in A. \quad (\text{infiltration rule})$$

An *infiltration algebra* $(A; 0, 1, +, *, (\delta_j)_{1 \leq j \leq d})$ is an algebra together with finitely many, not necessarily commuting infiltration functions $\delta_1, \dots, \delta_d : A \rightarrow A$. The product rule (*infiltration rule*) bears resemblance to that for endomorphisms (*endomorphism rule*) and for derivations (*derivation rule*). In the next section we clarify this connection, which will be exploited to simplify later proofs.

7.1.2 *σ -differential algebras.* Let $\sigma : A \rightarrow A$ be an endomorphism. A σ -*derivation* is a linear function $\delta : A \rightarrow A$ satisfying

$$\delta(\alpha * \beta) = \delta\alpha * \beta + \sigma\alpha * \delta\beta, \quad \text{for all } \alpha, \beta \in A. \quad (\sigma\text{-derivation rule})$$

If we take the identity endomorphism $\sigma = \text{id}$ then we recover the notion of derivation (*derivation rule*). We observe that infiltrations arise as σ -derivations of a special kind.

Lemma 27. *Let $\sigma, \delta : A \rightarrow A$ be two functions satisfying the equation*

$$\sigma\alpha = \alpha + \delta\alpha, \quad \text{for every } \alpha \in A. \quad (\sigma\text{-}\delta)$$

(1) *If σ is an endomorphism, then the unique δ satisfying $(\sigma\text{-}\delta)$ is an infiltration.*

We call δ the infiltration induced by σ .

(2) *If δ is an infiltration, then the unique σ satisfying $(\sigma\text{-}\delta)$ is an endomorphism.*

We call σ the endomorphism induced by δ .

In both cases, δ is a σ -derivation.

For instance, the identity endomorphism $\sigma = \text{id}$ induces the trivial infiltration $\delta\alpha = 0$ (and vice versa). We will use the lemma in the next section to extract an infiltration from an endomorphism, and conversely to extract an endomorphism from an infiltration (cf. the proof of Lemma 28).

PROOF. Regarding point (1), let σ be an endomorphism and consider the unique map δ satisfying $(\sigma\text{-}\delta)$. Linearity is clear. Regarding products, we have $\delta(\alpha \cdot \beta) = \sigma(\alpha \cdot \beta) - \alpha \cdot \beta = \sigma\alpha \cdot \sigma\beta - \alpha \cdot \beta = (\alpha + \delta\alpha) \cdot (\beta + \delta\beta) - \alpha \cdot \beta = \alpha \cdot \delta\beta + \delta\alpha \cdot \beta + \delta\alpha \cdot \delta\beta$. Regarding point (2), let δ be an infiltration and let σ be the unique map satisfying $(\sigma\text{-}\delta)$. Linearity is clear. Regarding products we have $\sigma(\alpha \cdot \beta) = \alpha \cdot \beta + \delta(\alpha \cdot \beta) = \alpha \cdot \beta + \delta\alpha \cdot \beta + \alpha \cdot \delta\beta + \delta\alpha \cdot \delta\beta = \sigma\alpha \cdot \beta + \sigma\alpha \cdot \delta\beta = \sigma\alpha \cdot \sigma\beta$. $\square$

7.1.3 *Infiltration algebras of polynomials.* As an example of an infiltration algebra, consider the algebra of polynomials $(\mathbb{Q}[X]; 0, 1, +, \cdot)$. An infiltration of this algebra is thus a linear function $\Delta : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ satisfying the product rule

$$\Delta(\alpha \cdot \beta) = \Delta\alpha \cdot \beta + \alpha \cdot \Delta\beta + \Delta\alpha \cdot \Delta\beta = \Delta\alpha \cdot \beta + S\alpha \cdot \Delta\beta,$$

where we have defined $S\alpha := \alpha + \Delta\alpha$. By Lemma 27, the function Δ is an infiltration iff S is an endomorphism. Thus the polynomial algebra with an infiltration Δ carries the structure of an S -differential algebra. We have seen that any function $\Delta : X \rightarrow \mathbb{Q}[X]$ extends uniquely to an endomorphism, resp., a derivation $\Delta : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$. We now state and prove the analogous property for infiltrations, which will be used later in the definition of infiltration automata.

Lemma 28. Any function $\Delta : X \rightarrow \mathbb{Q}[X]$ extends to a unique infiltration function $\Delta : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$.

PROOF. Consider the map $S : X \rightarrow \mathbb{Q}[X]$ s.t. $SX_i := X_i + \Delta X_i$ for all $1 \leq i \leq k$ and extend it to an endomorphism $S : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$. Now extend Δ to the whole polynomial algebra by letting $\Delta\alpha := S\alpha - \alpha$ for all $\alpha \in \mathbb{Q}[X]$. Since S is an endomorphism and the pair S, Δ by definition satisfies $(\sigma\text{-}\delta)$, by Lemma 27 we have that Δ is an infiltration function. Uniqueness can be shown by induction on the total degree of polynomials. We given another argument. Let $\Delta_1, \Delta_2 : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ be two infiltration functions extending $\Delta : X \rightarrow \mathbb{Q}[X]$. Then $S_1, S_2 : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$ defined by $S_i\alpha := \alpha + \Delta_i\alpha$ ($\alpha \in \mathbb{Q}[X]$, $i \in \{1, 2\}$) are endomorphisms by Lemma 27. But S_1, S_2 agree on X , and thus $S_1 = S_2$, implying $\alpha + \Delta_1\alpha = \alpha + \Delta_2\alpha$ ($\forall \alpha \in \mathbb{Q}[X]$) and thus $\Delta_1 = \Delta_2$. $\square$

7.1.4 Infiltration algebras of series. We show that series can be endowed with the structure of an infiltration algebra. While for polynomials the notion of product is fixed and we have used it to define infiltration functions, for series we proceed the other way around: The transition structure given by left derivatives is fixed and we define the infiltration product by demanding that left derivatives be infiltration functions w.r.t. this product. Formally, the *infiltration product* of two series $f, g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, denoted by $f \uparrow g$, is defined coinductively as follows:

$$\begin{aligned} [\varepsilon](f \uparrow g) &= f_\varepsilon \cdot g_\varepsilon, & (\uparrow\text{-}\varepsilon) \\ \delta_a^L(f \uparrow g) &= \delta_a^L f \uparrow g + f \uparrow \delta_a^L g + \delta_a^L f \uparrow \delta_a^L g, \quad \forall a \in \Sigma. & (\uparrow\text{-}\delta_a^L) \end{aligned}$$

To see why this uniquely defines a series $f \uparrow g$, we can reason by induction on the length of words. The first rule gives us the value for the constant term. For nonempty words, we have

$$[a \cdot w](f \uparrow g) = [w](\delta_a^L(f \uparrow g)) = [w](\delta_a^L f \uparrow g) + [w](f \uparrow \delta_a^L g) + [w](\delta_a^L f \uparrow \delta_a^L g),$$

where the three coefficients for word $w \in \Sigma^*$ are obtained by the inductive assumption. Infiltration product is an associative, commutative, and bilinear operation on series, with identity the series $1 \cdot \varepsilon$ (the same identity as for the shuffle product). We call the resulting structure $\mathbb{Q}\langle\langle\Sigma\rangle\rangle_\uparrow := (\mathbb{Q}\langle\langle\Sigma\rangle\rangle; 0, 1 \cdot \varepsilon, +, \uparrow)$ the *infiltration algebra of series*. Infiltration has been introduced by Chen, Fox, and Lyndon [ChenFoxLyndon:AM:1958] in the context of the *free differential calculus* of Fox [Fox:AM:1953]. More recently, it has been considered in a coalgebraic setting [BasoldHansenPinRutten:MCS:2017]. From a concurrency perspective, it models the *synchronising interleaving* where processes are allowed (but not forced) to jointly perform the input actions (thanks to the term $\delta_a^L f \uparrow \delta_a^L g$ in $(\uparrow\text{-}\delta_a^L)$). For instance, in the interleaving semantics we have $ab \sqcup a = 2aab + aba$, but in the synchronising interleaving we have $ab \uparrow a = 2aab + aba + ab$.

Remark 12. We have chosen a coinductive approach to the definition of infiltration product since it brings to the foreground the similarity with Hadamard and shuffle products. Alternatively, one can define the infiltration product on finite words by induction on their length, and then lift it to series by linearity and continuity [Lothaire:CUP:1997].

Remark 13 (Differential algebra of series). For every input symbol $a \in \Sigma$, consider the linear map $\sigma_a : \mathbb{Q}\langle\langle\Sigma\rangle\rangle \rightarrow \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ defined by

$$\sigma_a f := f + \delta_a^L f, \text{ for every } f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle. \quad (\sigma_a)$$

Sometimes we write σ_a^L to stress that the definition is w.r.t. the left derivative. Since δ_a^L is an infiltration function, thanks to Lemma 27 we have that σ_a is an infiltration algebra endomorphism, and thus δ_a^L is a σ_a -derivation. Using the

definition of σ_a , the rule $(\uparrow\text{-}\delta_a^L)$ can be rewritten more succinctly as follows:

$$\delta_a^L(f \uparrow g) = \delta_a^L f \uparrow g + \sigma_a f \uparrow \delta_a^L g \quad \forall a \in \Sigma. \quad (\uparrow\text{-}\delta_a^L)$$

This should be compared to the fact that δ_a^L is a derivation (i.e., for the identity endomorphism) of the shuffle algebra.

In the next lemma we observe that also right derivatives are infiltration functions. This shows that infiltration product can be defined equivalently from the left and from the right. This is a symmetry shared by the Hadamard and shuffle products, however there is no reason why it should hold for an arbitrary product. We will be using this property in the proof of closure under right derivatives for infiltration-finite series (Lemma 31). In the following, define $\sigma_a^R f := f + \delta_a^R f$, for all $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$.

Lemma 29. *The right derivative operator δ_a^R (with $a \in \Sigma$) is an infiltration function of the infiltration algebra: it is linear and it satisfies*

$$\delta_a^R(f \uparrow g) = \delta_a^R f \uparrow g + f \uparrow \delta_a^R g + \sigma_a^R f \uparrow \delta_a^R g = \delta_a^R f \uparrow g + \sigma_a^R f \uparrow \delta_a^R g, \quad \forall f, g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle. \quad (\uparrow\text{-}\delta_a^R)$$

PROOF. We proceed by coinduction. The constant terms of the two sides coincide:

$$\begin{aligned} [\varepsilon](\delta_a^R(f \uparrow g)) &= [\varepsilon](\delta_a^L(f \uparrow g)) = [\varepsilon](\delta_a^L f \uparrow g + f \uparrow \delta_a^L g + \delta_a^L f \uparrow \delta_a^L g) = f_a \cdot g_\varepsilon + f_\varepsilon \cdot g_a + f_a \cdot g_a = \\ &= [\varepsilon](\delta_a^R f \uparrow g + f \uparrow \delta_a^R g + \delta_a^R f \uparrow \delta_a^R g). \end{aligned}$$

Now consider an input symbol $b \in \Sigma$. Thanks to Lemma 3, we obtain the following commutation rules:

$$\delta_a^R \delta_b^L f = \delta_b^L \delta_a^R f, \quad \sigma_a^R \delta_b^L f = \delta_b^L \sigma_a^R f, \quad \delta_a^R \sigma_b^L f = \sigma_b^L \delta_a^R f, \quad \text{and} \quad \sigma_a^R \sigma_b^L f = \sigma_b^L \sigma_a^R f, \quad \forall f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle. \quad (18)$$

Thanks to the rules above, we have

$$\begin{aligned} \delta_b^L \delta_a^R(f \uparrow g) &= \delta_a^R \delta_b^L(f \uparrow g) = && \text{by (18)} \\ &= \delta_a^R(\delta_b^L f \uparrow g + \sigma_b^L f \uparrow \delta_b^L g) = && \text{(def. } \uparrow) \\ &= \delta_a^R(\delta_b^L f \uparrow g) + \delta_a^R(\sigma_b^L f \uparrow \delta_b^L g) = && \text{(linearity of } \delta_a^R) \\ &= \delta_a^R \delta_b^L f \uparrow g + \sigma_a^R \delta_b^L f \uparrow \delta_a^R g + \delta_a^R \sigma_b^L f \uparrow \delta_b^L g + \sigma_a^R \sigma_b^L f \uparrow \delta_a^R \delta_b^L g = && \text{(coinduction)} \\ &= \delta_b^L \delta_a^R f \uparrow g + \delta_b^L \sigma_a^R f \uparrow \delta_a^R g + \sigma_b^L \delta_a^R f \uparrow \delta_b^L g + \sigma_b^L \sigma_a^R f \uparrow \delta_b^L \delta_a^R g = && \text{by (18)} \\ &= \delta_b^L(\delta_a^R f \uparrow g) + \delta_b^L(\sigma_a^R f \uparrow \delta_a^R g) && \text{(def. } \uparrow) \\ &= \delta_b^L(\delta_a^R f \uparrow g + \sigma_a^R f \uparrow \delta_a^R g). && \text{(linearity of } \delta_b^L) \quad \square \end{aligned}$$

7.2 Infiltration-finite series

We have seen that the Hadamard and shuffle products induce corresponding classes of Hadamard-finite (§ 5.2), resp. shuffle-finite series (§ 6.2.2), together with natural closure properties. The infiltration product admits an analogous development. For series $f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$, we denote by $\mathbb{Q}[f_1, \dots, f_k]_\uparrow$ the infiltration subalgebra finitely generated by the f_i 's. An infiltration algebra is *differential* if it is closed under δ_a^L , for all $a \in \Sigma$.

Definition 2. A series is *infiltration finite* if it belongs to a finitely generated differential infiltration algebra of series.

The following lemma unpacks the definition above and provides our working definition for infiltration finite series. Its statement and proof are similar to Lemma 12: We present the former but we omit the latter.

Lemma 30. *A series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is infiltration finite iff there are generators $f_1, \dots, f_k \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ s.t.:*

- (1) $f \in \mathbb{Q}[f_1, \dots, f_k]_{\uparrow}$ and
- (2) $\delta_a^L f_i \in \mathbb{Q}[f_1, \dots, f_k]_{\uparrow}$ for every $a \in \Sigma$ and $1 \leq i \leq k$.

Moreover, we can assume w.l.o.g. that $f = f_1$.

Infiltration-finite series have several pleasant closure properties.

Lemma 31. *The class of infiltration-finite series is an effective differential infiltration algebra: It contains $\mathbb{0}$, $1 \cdot \varepsilon$, and it is effectively closed under scalar product $c \cdot f$, sum $f + g$, infiltration product $f \uparrow g$, and left derivative $\delta_a^L f$. Moreover,*

- if f is infiltration finite, then $\delta_a^R f$ is effectively infiltration finite, and
- If $g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is an anti-derivative of a tuple of infiltration-finite series $f = (f_a)_{a \in \Sigma} \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle^{\Sigma}$, then g is effectively infiltration finite.

PROOF. All closure properties are based on simple manipulations of the generators. They are based on Lemma 30 and follow the same pattern as for Hadamard-finite and shuffle-finite series (cf. Lemma 13). As an illustration, we show closure under right derivatives. Let f be an infiltration-finite series and fix an input symbol $a \in \Sigma$. By Lemma 30, there are generators $f_1, \dots, f_k$ with $f = f_1$ generating the δ^L -closed infiltration algebra $A := \mathbb{Q}[f_1, \dots, f_k]_{\uparrow}$. We adjoin new generators $\delta_a^R f_1, \dots, \delta_a^R f_k$, obtaining the larger infiltration algebra $B := \mathbb{Q}[f_1, \dots, f_k, \delta_a^R f_1, \dots, \delta_a^R f_k]_{\uparrow}$. By construction, $\delta_a^R f = \delta_a^R f_1$ is in B . It remains to show that B is δ^L -closed. Since A is δ^L -closed, it suffices to show this for the new generators. So consider $\delta_b^L \delta_a^R f_i$ for arbitrary $a, b \in \Sigma$ and $1 \leq i \leq k$. By Lemma 3, left and right derivatives commute, so we have $\delta_b^L \delta_a^R f_i = \delta_a^R \delta_b^L f_i$. Since A is δ^L -closed, $\delta_b^L f_i$ is in A , and thus we can write $\delta_b^L f_i = p_i^b(f_1, \dots, f_k)$ for some polynomial p_i^b (where product is interpreted as infiltration product). Thanks to linearity of right derivative and Lemma 29, a proof by structural induction on polynomials shows that $\delta_a^R(p_i^b(f_1, \dots, f_k))$ can be written as a polynomial expression $q_i^{a,b}(f_1, \dots, f_k, \delta_a^R f_1, \dots, \delta_a^R f_k)$. We thus have $\delta_b^L \delta_a^R f_i = \delta_a^R \delta_b^L f_i = q_i^{a,b}(f_1, \dots, f_k, \delta_a^R f_1, \dots, \delta_a^R f_k) \in B$, as required. $\square$

In this section we have defined the class of infiltration-finite series in a semantic fashion, based on which we have shown effective closure properties (Lemma 31). In order to show that infiltration-finite series constitute an effective prevariety, it remains to show that the zeroness problem is decidable for them. To this end, we introduce in the next section a suitable automaton model capturing the same class.

7.3 Infiltration automata

The syntax and semantics of infiltration automata is largely parallel to that of Hadamard and shuffle automata. An *infiltration automaton* is a tuple $\mathcal{A} = (\Sigma, X, F, \Delta)$ where Σ is a finite *input alphabet*, $X = \{X_1, \dots, X_k\}$ is a finite set of *nonterminal symbols*, $F : X \rightarrow \mathbb{Q}$ is the *output function*, and $\Delta : \Sigma \times X \rightarrow \mathbb{Q}[X]$ is the *transition function*. The only difference is how to extend the transition function $\Delta_a : X \rightarrow \mathbb{Q}[X]$ from variables X to configurations $\mathbb{Q}[X]$. Here, we extend Δ_a to a unique infiltration $\Delta_a : \mathbb{Q}[X] \rightarrow \mathbb{Q}[X]$, thanks to Lemma 28. In a way entirely parallel to Hadamard and shuffle automata, this defines a semantic function $\mathcal{A}[\llbracket _ \rrbracket] : \mathbb{Q}[X] \rightarrow \mathbb{Q}\langle\langle\Sigma\rangle\rangle$. When the automaton $\mathcal{A}$ is fixed by the context, we also just write $\llbracket _ \rrbracket$. The main property of the semantics is that it is a homomorphism, in complete analogy for the corresponding properties for Hadamard and shuffle automata (Lemma 14, resp., Lemma 20).

Lemma 32 (Properties of the semantics). *The semantic function $\mathcal{A}[_]$ is a homomorphism from the S -differential algebra of polynomials to the σ -differential infiltration algebra of series. In other words, for every configurations $\alpha, \beta \in \mathbb{Q}[X]$,*

$$\mathcal{A}[c \cdot \alpha] = c \cdot \mathcal{A}[\alpha], \quad \forall c \in \mathbb{Q}, \quad (19)$$

$$\mathcal{A}[\alpha + \beta] = \mathcal{A}[\alpha] + \mathcal{A}[\beta], \quad (20)$$

$$\mathcal{A}[\alpha \cdot \beta] = \mathcal{A}[\alpha] \uparrow \mathcal{A}[\beta], \quad (21)$$

$$\mathcal{A}[S_a \alpha] = \sigma_a(\mathcal{A}[\alpha]), \quad \forall a \in \Sigma \quad (22)$$

$$\mathcal{A}[\Delta_a \alpha] = \delta_a^L(\mathcal{A}[\alpha]), \quad \forall a \in \Sigma. \quad (23)$$

PROOF. Property (23) holds by definition of the semantics. Properties (19) and (20) follow from linearity of δ_a^L and Δ_a . From these, (22) follows easily: $\llbracket S_a \alpha \rrbracket = \llbracket \alpha + \Delta_a \alpha \rrbracket = \llbracket \alpha \rrbracket + \llbracket \Delta_a \alpha \rrbracket = \llbracket \alpha \rrbracket + \delta_a^L \llbracket \alpha \rrbracket = \sigma_a \llbracket \alpha \rrbracket$. For the last property (21), we proceed coinductively. Clearly, the constant term of $\llbracket \alpha \cdot \beta \rrbracket$ and $\llbracket \alpha \rrbracket \uparrow \llbracket \beta \rrbracket$ is the same, namely $F\alpha \cdot F\beta$. For the coinductive step, we show that the left derivatives of $\llbracket \alpha \cdot \beta \rrbracket$ and $\llbracket \alpha \rrbracket \uparrow \llbracket \beta \rrbracket$ coincide:

$$\begin{aligned} \delta_a^L \llbracket \alpha \cdot \beta \rrbracket &= \llbracket \Delta_a(\alpha \cdot \beta) \rrbracket = \llbracket \Delta_a \alpha \cdot \beta + S_a \alpha \cdot \Delta_a \beta \rrbracket = \llbracket \Delta_a \alpha \cdot \beta \rrbracket + \llbracket S_a \alpha \cdot \Delta_a \beta \rrbracket \stackrel{!}{=} \llbracket \Delta_a \alpha \rrbracket \uparrow \llbracket \beta \rrbracket + \llbracket S_a \alpha \rrbracket \uparrow \llbracket \Delta_a \beta \rrbracket = \\ &= (\delta_a^L \llbracket \alpha \rrbracket) \uparrow \llbracket \beta \rrbracket + (\sigma_a \llbracket \alpha \rrbracket) \uparrow \delta_a^L \llbracket \beta \rrbracket = \delta_a^L(\llbracket \alpha \rrbracket \uparrow \llbracket \beta \rrbracket). \end{aligned} \quad \square$$

The following characterisation connects infiltration automata and infiltration-finite series. Thanks to it, we can use algorithmic results on infiltration automata for infiltration-finite series. We omit the proof, which is analogous to that of Lemma 15, relying on Lemmas 30 and 32.

Lemma 33. *A series is recognised by an infiltration automaton if, and only if, it is infiltration finite.*

7.4 Equality problem

We show that the equality problem for infiltration-finite series is decidable. To this end, we leverage a classic technique based on chains of polynomial ideals and Hilbert's finite basis theorem. This technique has been used by Novikov and Yakovenko to derive decidability results on continuous time and discrete time dynamical systems [NovikovYakovenko:1999]. In the terminology of this paper, their results yield decidability of equality for univariate ($\Sigma = \{a\}$) Hadamard-finite and shuffle-finite series. It has provided the basis of several subsequent decidability results, such as in the work of Benedikt *et al.* for multivariate ($|\Sigma| \geq 1$) Hadamard-finite series (a.k.a. polynomial automata [BenediktDuffSharadWorrell:PolyAut:LICS:2017]), in our work on multivariate shuffle-finite series [Clemente:CONCUR:2024], and in the work of Boreale *et al.* for univariate infiltration-finite series [BorealeGorla:CONCUR:2021, BorealeCollodiGorla:ACMTCL:2024].

Fix an infiltration automaton $\mathcal{A} = (\Sigma, X, F, \Delta)$ recognising an infiltration-finite series $f = \mathcal{A}[\llbracket X_1 \rrbracket] \in \mathbb{Q}\langle\langle \Sigma \rangle\rangle$. Since equality reduces to zeroness, we show how to decide $\mathcal{A}[\llbracket X_1 \rrbracket] = 0$. For a set of configurations $P \subseteq \mathbb{Q}[X]$, let the *polynomial ideal generated by P* , written $\langle P \rangle$, be the set of all polynomials of the form $\beta_1 \cdot \alpha_1 + \dots + \beta_n \cdot \alpha_n$, with $\alpha_1, \dots, \alpha_n \in P$ configurations in P and $\beta_1, \dots, \beta_n \in \mathbb{Q}[X]$ arbitrary configurations. (We refer to [CoxLittleOShea:Ideals:2015] for more details on algebraic geometry.) We will be looking at the ideal $\langle C \rangle$ generated by all reachable configurations $C := \{\Delta_w X_1 \in \mathbb{Q}[X] \mid w \in \Sigma^*\}$. The intuition is that, while the set of reachable configurations C is complicated (formally, its membership problem is undecidable), the ideal it generates $\langle C \rangle$ is simple: It is finitely generated (as an ideal) and we can compute a finite set of generators for it. In other words, $\langle C \rangle$ is effectively finitely generated. Moreover, the ideal

abstraction is sound for the zeroness problem: when the generators vanish, so do all polynomials in the ideal they generate, and so no information is lost.

For a length $n \in \mathbb{N}$, consider the polynomial ideal I_n generated by all configurations reachable from X_1 by reading words of length $\leq n$:

$$I_n := \langle \Delta_w X_1 \mid w \in \Sigma^{\leq n} \rangle.$$

The following lemma describes the relevant properties of these ideals.

Lemma 34. (1) $I_n \subseteq I_{n+1}$.

(2) $\Delta_a I_n \subseteq I_{n+1}$.

(3) $I_{n+1} = I_n + \langle \Delta_a I_n \mid a \in \Sigma \rangle$.

(4) $I_n = I_{n+1}$ implies $I_n = I_{n+1} = I_{n+2} = \dots$.

PROOF. The first point holds since the operation of constructing the ideal generated by a set is monotonic for inclusion of sets. To prove the second point we use the fact that Δ_a is an infiltration (this is the only property relying directly on this). Let $\alpha \in \Delta_a I_n$. Then α is of the form

$$\alpha = \Delta_a \sum_{i=1}^m \beta_i \cdot \Delta_{w_i} X_1 = \sum_{i=1}^m \Delta_a (\beta_i \cdot \Delta_{w_i} X_1) = \sum_{i=1}^m (\underbrace{\Delta_a \beta_i \cdot \Delta_{w_i} X_1}_{\in I_n} + \underbrace{\Delta_a \beta_i \cdot \Delta_a \Delta_{w_i} X_1}_{\in I_{n+1}}),$$

which is clearly in I_{n+1} . We now consider the third point. The “ $\supseteq$ ” inclusion holds by the first two points and the fact that I_{n+1} is an ideal. For the other inclusion, let $\alpha \in I_{n+1}$. We can write α separating words of length exactly $n+1$ from the rest, and we have

$$\alpha = \underbrace{\beta}_{\in I_n} + \sum_{w \in \Sigma^{n+1}} \beta_w \cdot \Delta_w X_1 = \beta + \sum_{a \in \Sigma} \sum_{w \in \Sigma^n} \beta_{w \cdot a} \cdot \Delta_{w \cdot a} X_1 = \beta + \sum_{a \in \Sigma} \sum_{w \in \Sigma^n} \beta_{w \cdot a} \cdot \Delta_a \underbrace{\Delta_w X_1}_{\in I_n},$$

where the latter quantity is clearly in the ideal generated by $\Delta_a I_n$. The last point follows from the previous one since we have shown that I_{n+1} is a function of I_n . $\square$

Now consider the chain of ideals

$$I_0 \subseteq I_1 \subseteq \dots \subseteq \mathbb{Q}[X]. \quad (24)$$

By Hilbert’s finite basis theorem [CoxLittleOShea:Ideals:2015], there is $N \in \mathbb{N}$ s.t. the chain stabilises $I_N = I_{N+1} = \dots$. Thanks to the last point of Lemma 34, such an N is computable: Indeed, N can be taken to be the smallest $n \in \mathbb{N}$ s.t. $I_n = I_{n+1}$, and ideal equality can be decided by checking that the generators of I_{n+1} are all in I_n . The latter test is an instance of the *ideal membership problem*, which is decidable (in exponential space) [Mayr:STACS:1989]. We call such minimal N the *stabilisation index* of the ideal chain. We remark that it is crucial that such an N not only exists (as follows already from Hilbert’s theorem, whose proof is notoriously non-constructive), but it can be computed. The following property provides a decidable criterion for zeroness.

Lemma 35. Let $N \in \mathbb{N}$ be the stabilisation index of the ideal chain (24). We then have

$$\mathcal{A}[\![X_1]\!] = 0 \quad \text{if, and only if,} \quad [w](\mathcal{A}[\![X_1]\!]) = 0 \text{ for all } w \in \Sigma^{\leq N}.$$

PROOF. The “only if” direction is trivial. For the other direction, assume that the semantics is zero for words of length $\leq N$. Thanks to the properties of the semantics (Lemma 32), for every word $w \in \Sigma^*$ we can write

$$\delta_w^L[X_1] = \llbracket \Delta_w X_1 \rrbracket = \llbracket \sum_{u \in \Sigma^{\leq N}} \beta_u \cdot \Delta_u X_1 \rrbracket = \sum_{u \in \Sigma^{\leq N}} \llbracket \beta_u \rrbracket \uparrow \llbracket \Delta_u X_1 \rrbracket,$$

where we have used the fact that $\Delta_w X_1$ is in I_N . By taking constant terms on both sides, we have

$$[w][X_1] = [\varepsilon](\delta_w^L[X_1]) = \sum_{u \in \Sigma^{\leq N}} [\varepsilon][\beta_u] \cdot [\varepsilon][\Delta_u X_1] = \sum_{u \in \Sigma^{\leq N}} [\varepsilon][\beta_u] \cdot [\varepsilon](\delta_u^L[X_1]) = \sum_{u \in \Sigma^{\leq N}} [\varepsilon][\beta_u] \cdot [u][X_1].$$

But u has length $\leq N$, and thus by the assumption $[u][X_1] = 0$, implying $[w][X_1] = 0$, as required. $\square$

Theorem 36. *The equality problem for infiltration-finite series is decidable.*

PROOF. By using the effective closure properties of infiltration-finite series, we reduce equality $f = g$ to zeroness $f - g = 0$. So let $f = \mathcal{A}[X_1]$ be an infiltration-finite series, recognised by an infiltration automaton $\mathcal{A}$. Compute the stabilisation index $N \in \mathbb{N}$ of the ideal chain (24). Thanks to Lemma 35, we can decide $f = 0$ by enumerating all words w of length $\leq N$ and checking $[w]f = 0$. The latter test is performed by applying the semantics of the infiltration automaton. $\square$

Remark 14 (Effective algebraic variety of initial conditions). The zeroness algorithm above solves a more general problem. Not only it decides whether a given initial condition gives rise to the zero semantics, but it also shows that the set of all such initial conditions forms an *effective algebraic variety* (in the sense of algebraic geometry) and computes a finite representation for it in the form of a finite set of polynomials, as we now explain. In fact, the same remark applies to equivalence and commutativity, and to Hadamard and shuffle automata.

Given any set of polynomials $P \subseteq \mathbb{Q}[X]$, let their *zero set* $V(P) \subseteq \mathbb{C}^k$ be the set of their common zeroes,

$$V(P) := \{x \in \mathbb{C}^k \mid \forall p \in P : p(x) = 0\}.$$

Sets of the form $V(P) \subseteq \mathbb{C}^k$ for some $P \subseteq \mathbb{Q}[X]$ are called *algebraic varieties* [CoxLittleOShea:Ideals:2015] and play a prominent role in algebraic geometry. Fix a Hadamard, shuffle, or infiltration automaton $\mathcal{A}_F = (\Sigma, X, F, \Delta)$, where we have made explicit the dependency of the automaton on the initial condition F . In the algebraic framework, it is more natural to consider a slightly more general class of initial conditions $F : X \rightarrow \mathbb{C}$, mapping nonterminals to complex numbers. For every initial configuration $\alpha \in \mathbb{Q}[X]$, let $\mathcal{F}_\alpha$ be the set of initial conditions giving rise to the zero semantics from α ,

$$\mathcal{F}_\alpha := \{F : X \rightarrow \mathbb{C} \mid \mathcal{A}_F[\alpha] = 0\}.$$

Thus, by definition $\mathcal{F}_\alpha$ is the set of common zeroes $\mathcal{F}_\alpha = V(P)$ of the (possibly infinite) set of polynomials

$$P := \{\Delta_w \alpha \in \mathbb{Q}[X] \mid w \in \Sigma^*\},$$

and thus $\mathcal{F}_\alpha$ is an algebraic variety. The zeroness problem amounts to decide whether a given initial condition F is in this algebraic variety, which is just an instance of the membership problem. In fact, we can replace P by the polynomial ideal it generates, and obtain that $\mathcal{F}_\alpha = V(P) = V(\langle P \rangle)$ is also the set of common zeroes of $\langle P \rangle$. But $\langle P \rangle$ is just the limit $\bigcup_{n=0}^{\infty} I_n$ of the polynomial ideal chain (24). We know that there is a computable $N \in \mathbb{N}$ s.t. $\langle P \rangle = I_N = \langle P_N \rangle$, and thus $\mathcal{F}_\alpha$ is the set of common zeroes $\mathcal{F}_\alpha = V(P_N)$ of the *finite* and *computable* set of polynomials $P_N \subseteq P$ obtained by

considering words of length $\leq N$,

$$P_N := \{ \Delta_w \alpha \in \mathbb{Q}[X] \mid w \in \Sigma^{\leq N} \}.$$

As a consequence, $\mathcal{F}_\alpha$ is an *effective* algebraic variety, since its membership problem $F \in \mathcal{F}_\alpha$ is decidable: it suffices to check whether all polynomials in P_N vanish on F . We can use P_N to solve the following two variants of the zeroness problem:

- (1) Decide whether *there exists* an initial condition $F : X \rightarrow \mathbb{C}$ giving rise to the zero semantics.
Thanks to *Hilbert's Nullstellensatz* [CoxLittleOShea:Ideals:2015], this amounts to check whether the ideal generated by P_N does not contain 1. The latter can be established with standard techniques from computational algebraic geometry, such as *Gröbner bases* [CoxLittleOShea:Ideals:2015].
- (2) Decide whether *every* initial condition $F : X \rightarrow \mathbb{C}$ gives rise to the zero semantics. This is equivalent to $P_N = \{0\}$.

The following theorem summarises the results on infiltration-finite series that we have obtained.

Theorem 37. *The class of infiltration-finite series is an effective prevariety. Consequently, the commutativity problem is decidable. Finally, the commutativity problem is at least as hard as the equivalence problem (under polynomial time reductions)*

PROOF. The effective closure properties (V.1) and (V.2) have been established in Lemma 31 and decidability of equality in Theorem 36. Decidability of commutativity follows from Theorem 1. Hardness of commutativity follows from Lemma 5 and the fact that infiltration-finite series are effectively closed under anti-derivatives (Lemma 31). $\square$

Acknowledgments

We warmly thank Arka Ghosh, Aliaume Lopez, and Filip Mazowiecki for valuable discussions during the preparation of this work.

A Polynomial vs. Hadamard automata

In this section, we give more details about the relationship between polynomial and Hadamard automata, expanding on Remark 7. A *polynomial automaton* [BenediktDuffSharadWorrell:PolyAut:LICS:2017] is a tuple $\mathcal{A} = (k, \Sigma, Q, q_I, \Delta, F)$ where $k \in \mathbb{N}$ is the *dimension*, Σ is a finite *input alphabet*, $Q = \mathbb{Q}^k$ is the set of *states*, $q_I \in Q$ is the *initial state*, $\Delta : \Sigma \rightarrow Q \rightarrow Q$ is the *polynomial transition function*, and $F : Q \rightarrow \mathbb{Q}$ is the *polynomial output function*. For every input symbol $a \in \Sigma$, the polynomial map $\Delta^a : Q^k \rightarrow Q^k$ is presented as a tuple of polynomials $\Delta^a = (\Delta_1^a, \dots, \Delta_k^a) \in \mathbb{Q}[y_1, \dots, y_k]^k$, inducing a polynomial action on states

$$q \cdot a := \Delta^a(q) = (\Delta_1^a(q), \dots, \Delta_k^a(q)) \in Q, \quad \text{for all } q \in Q.$$

Similarly, the polynomial output function F is presented as a polynomial $F \in \mathbb{Q}[y_1, \dots, y_k]$ inducing the corresponding polynomial map $F(q) = F(q_1, \dots, q_k) \in \mathbb{Q}$ for every $q = (q_1, \dots, q_k) \in Q$. The action of Σ is extended to words $w \in \Sigma^*$ homomorphically: $q \cdot \varepsilon := q$ and $q \cdot (a \cdot w) := (q \cdot a) \cdot w$. The *semantics* of a state $q \in Q$ of a polynomial automaton $\mathcal{A}$ is the series defined as follows:

$$\begin{aligned} \mathcal{A}[[q]] &\in \mathbb{Q}\langle\langle \Sigma \rangle\rangle \\ [w](\mathcal{A}[[q]]) &:= F(q \cdot w), \quad \text{for every } w \in \Sigma^*. \end{aligned}$$

The semantics of the automaton $\mathcal{A}$ is the series recognised by the initial state $\mathcal{A}[\![q_I]\!]$. The following lemma shows that polynomial automata recognise precisely the Hadamard-finite series, up to reversal of the input. Since Hadamard-finite series coincide with the series recognised by Hadamard automata (Lemma 15), the same holds for Hadamard automata.

Lemma 38. *A series $f \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ is recognisable by a polynomial automaton if, and only if, its reversal f^R is Hadamard finite (equivalently, recognised by a Hadamard automaton).*

Recall that f^R is the series that maps a word $w \in \Sigma^*$ to $f(w^R)$, where $w^R = a_n \cdots a_1$ is the reversal of $w = a_1 \cdots a_n$.

PROOF. For the “only if” direction, let $\mathcal{A}$ be a polynomial automaton recognising the series $f = \mathcal{A}[\![q_I]\!]$. For every $1 \leq i \leq k$, consider the series $g_i \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ defined as

$$g_i(w) := \pi_i(q_I \cdot w^R), \quad \text{for all } w \in \Sigma^*.$$

In other words, $g_i(w)$ is the value of component i after reading w^R from the initial state. Consider now the Hadamard algebra

$$A := \mathbb{Q}[g_1, \dots, g_k]_{\odot} \subseteq \mathbb{Q}\langle\langle\Sigma\rangle\rangle.$$

Thanks to the working characterisation of Hadamard-finite series from Lemma 12, this part of the proof is concluded by the following two claims.

Claim. f^R is in A .

PROOF. By definition of f , we have $f = \mathcal{A}[\![q_I]\!]$. Consequently for every $w \in \Sigma^*$ we have

$$f^R(w) = F(q_I \cdot w^R) = F(\pi_1(q_I \cdot w^R), \dots, \pi_k(q_I \cdot w^R)) = F(g_1(w), \dots, g_k(w)),$$

and thus $f^R = F(g_1, \dots, g_k)$ is in A , as required. $\square$

Claim. For every $a \in \Sigma$ and $1 \leq i \leq k$, we have $\delta_a^L g_i \in A$.

PROOF. By the definition of g_i , for every $w \in \Sigma^*$ we have

$$(\delta_a^L g_i)(w) = g_i(a \cdot w) = \pi_i(q_I \cdot (a \cdot w)^R) = \pi_i(q_I \cdot (w^R \cdot a)) = \pi_i((q_I \cdot w^R) \cdot a) = \Delta_i^a(q_I \cdot w^R) = \Delta_i^a(g_1(w), \dots, g_k(w)),$$

and thus $\delta_a^L g_i = \Delta_i^a(g_1, \dots, g_k)$ is in A , as required. $\square$

For the “if” direction, let $g \in \mathbb{Q}\langle\langle\Sigma\rangle\rangle$ belong to a finitely generated Hadamard algebra $A := \mathbb{Q}[g_1, \dots, g_k]_{\odot}$ closed under left derivatives δ_a^L (with $a \in \Sigma$). Since $g \in A$, we can write $g = F(g_1, \dots, g_k)$ for some polynomial $F \in \mathbb{Q}[y_1, \dots, y_k]$. For every $1 \leq i \leq k$ and $a \in \Sigma$, the series $\delta_a^L g_i$ is in A and thus it can be written as $\Delta_i^a(g_1, \dots, g_k)$ for some polynomial $\Delta_i^a \in \mathbb{Q}[y_1, \dots, y_k]$. Finally, let $q_I := (g_1(\varepsilon), \dots, g_k(\varepsilon)) \in \mathbb{Q}^k$ be the tuple of constant terms of the generators. This provides us with the data required to construct a dimension k polynomial automaton $\mathcal{A} = (k, \Sigma, Q, q_I, \Delta, F)$. The proof is concluded with the following two claims.

Claim. For every $1 \leq i \leq k$ and $w \in \Sigma^*$, we have

$$g_i(w) = \pi_i(q_I \cdot w^R).$$

PROOF. We proceed by induction on w . In the base case $w = \varepsilon$, by definition we have $\pi_i(q_I \cdot \varepsilon^R) = \pi_i(q_I) = g_i(\varepsilon)$. In the inductive case, we have

$$\begin{aligned} g_i(a \cdot w) &= [w](\delta_a^L g_i) = [w](\Delta_i^a(g_1, \dots, g_k)) = \Delta_i^a(g_1(w), \dots, g_k(w)) = \Delta_i^a(\pi_1(q_I \cdot w^R), \dots, \pi_k(q_I \cdot w^R)) = \Delta_i^a(q_I \cdot w^R) = \\ &= \pi_i((q_I \cdot w^R) \cdot a) = \pi_i(q_I \cdot (w^R \cdot a)) = \pi_i(q_I \cdot (a \cdot w)^R). \end{aligned} \quad \square$$

Claim. We have $g = \llbracket q_I \rrbracket^R$.

PROOF. Consider an arbitrary $w \in \Sigma^*$, and we need to show $g(w) = F(q_I \cdot w^R)$. Thanks to the previous claim, we have $g(w) = F(g_1(w), \dots, g_k(w)) = F(\pi_1(q_I \cdot w^R), \dots, \pi_k(q_I \cdot w^R)) = F(q_I \cdot w^R)$. $\square$

Received XX August 2025